S Y N C D U B

The Founder Operating System

The internal handbook — how we think, engineer, research, build product, and win.

AI Video Localization · Indic-first

July 2026 · Living document — the canonical version lives in `docs/founder-os/`

Contents & How to Read

The internal handbook of SyncDub — how we think, engineer, research, build product, and win.

This is the document every SyncDub employee reads before writing their first line of code. It is not documentation of what exists; it is the operating system that decides what gets built, how, and why. It is written for a team of one today and a company of five hundred later — the principles hold at both sizes, and each chapter says explicitly what applies now versus at scale.

Every chapter is grounded in the real SyncDub codebase and its real gaps, not in generic startup advice. When a chapter cites `backend/server.py` or the Wav2Lip license, go look — the handbook earns trust by never pretending the current system is more than it is.

How to read this handbook

- **New engineer?** Read Part 0, then Part II, then Chapter 16 (Evaluation OS). Everything else as needed.
- **Making a product decision?** Chapter 3 (Decision Frameworks) is the gate; Part IV is the context.
- **Evaluating a new model or vendor?** Chapters 7, 15, and 16, in that order.
- **Founder, quarterly?** Re-read Chapters 2, 4, and 24, and answer every "Questions founders should ask" section honestly.

Table of Contents

Part III — AI Research OS: How AI Systems Think

Part IV — Product OS: How Product Decisions Are Made

Part V — Business OS: How the Company Wins

Appendices

This handbook is a living document. Chapters change by pull request, like code. If reality and the handbook disagree, fix one of them — never let them quietly diverge.

PART 0 — GROUND TRUTH

Chapter 1: The State of SyncDub Today

Every handbook that skips the honest audit becomes fiction. This chapter records exactly what SyncDub is on the day this handbook was written: what works, what is demo-grade, and which parts of the foundation cannot legally carry a company. Every later chapter builds on this baseline. When the system changes, change this chapter in the same pull request.

1.1 What SyncDub is right now

SyncDub is a working end-to-end pipeline that takes a Hindi video and produces a Marathi-dubbed, lip-synced video. That sentence is worth more than most pitch decks, because the pipeline actually runs:

1. **Upload** — a React frontend (`frontend/src/App.js`) posts a video to a FastAPI backend (`backend/server.py`).
2. **Audio extraction** — MoviePy pulls the audio track (`extract_audio` in `backend/inference_marathi.py`).
3. **Transcription** — OpenAI Whisper (medium, ~1.5 GB), forced to Hindi with `beam_size=2` as a deliberate speed/accuracy trade-off.
4. **Translation** — `deep-translator`'s GoogleTranslator wrapper, Hindi→Marathi, with segments grouped into context blocks rather than translated line-by-line.
5. **Speaker profiling** — Librosa pitch analysis; median frequency above ~160 Hz selects a female voice, otherwise male.
6. **Speech synthesis** — two paths: Edge-TTS neural voices (`mr-IN-AarohiNeural` / `mr-IN-ManoharNeural`), or XTTS v2 voice cloning via `backend/clone_bridge.py`, a subprocess bridge into a separate Python environment (`USE_VOICE_CLONING = True` in `inference_marathi.py`).
7. **Lip sync** — Wav2Lip, vendored wholesale in `backend/Wav2Lip_repo/`.
8. **Render** — MoviePy merges and writes `outputs/final_dubbed.mp4`.

Progress is reported through a single `progress.json` file written atomically (write-to-temp then `os.replace`) and polled by the frontend.

What is genuinely good in this code

Do not let the audit below obscure this: the codebase shows correct production instincts.

- **Context-block translation.** Grouping ASR segments before translating preserves Marathi grammar. Most first attempts translate line-by-line and produce garbage. Ours doesn't.
- **Lazy model loading and explicit unloading.** `get_whisper_model()` / `unload_models()` in `inference_marathi.py` manage memory deliberately, including

`torch.cuda.empty_cache()`. This is someone who has hit OOM in practice.

- **Corruption defense.** The Whisper loader checks for a truncated model download and deletes it rather than crashing cryptically.
- **Atomic progress writes.** The temp-file-then-rename pattern prevents the frontend from reading half-written JSON.
- **Environment isolation via bridge.** `clone_bridge.py` runs XTTS v2 in a separate Python env through a subprocess because its dependencies conflict with the main env. It is ugly and it works, which is the right order to solve problems in.

These instincts are the seed of our engineering culture. Chapter 7 turns them into principles.

1.2 What is demo-grade

The backend is a competition demo, not a service. This is not a criticism — it won the job it was hired for (see `SyncDub_Competition_Guide.md`) — but nobody should mistake it for a product backend.

Reality in the code	Consequence
One global `progress.json`, no job IDs	Two concurrent users corrupt each other's progress. The system is single-tenant by construction.
`subprocess.Popen` fire-and-forget in `/upload-video/`	No retries, no failure states, no way to know a job died except reading `pipeline.log`.
No authentication, CORS `allow_origins=["*"]`	Anyone who can reach the server can submit jobs and fetch outputs.
Output always `outputs/final_dubbed.mp4`	Every job overwrites the previous user's video.
Hardcoded `D:/Miniconda3/...ffmpeg.exe` path	The pipeline runs on exactly one Windows machine.
Uploaded filename used directly as the save path	Path traversal risk and collision risk in one line.
`test.py`, `test_tts.py`, `test_upload.py`	Scratch scripts, not a test suite. There is no CI and nothing for CI to run.
No `LICENSE` file in the repo	Our own legal posture is undefined, before we even discuss dependencies.

Chapter 8 defines the migration path from this to a real service. Chapter 14 defines when each item above *must* be paid down (trigger-based, not calendar-based).

1.3 The licensing reality — read this section twice

Not one component of the current AI pipeline can legally or reliably power a commercial product. This is the single most important fact in this handbook.

- **Wav2Lip** (`backend/Wav2Lip_repo/README.md`, verbatim): *"This repository can only be used for personal/research/non-commercial purposes."* Worse: its authors commercialized the technology as **Sync Labs** — our core lip-sync dependency is owned by a direct competitor, who profits from our category and controls our license.
- **XTTS v2 / Coqui** (used via `clone_bridge.py`): released under the Coqui Public Model License, **non-commercial**. Coqui the company has shut down, so there is no counterparty to buy a commercial license from.
- **deep-translator → Google Translate**: an unofficial wrapper around the free web endpoint. It violates Google's terms at commercial scale and can be rate-limited or blocked without notice.
- **Edge-TTS**: a reverse-engineered Microsoft endpoint, not a contracted API. It can break or be blocked any day, and offers no SLA, no invoice, and no legal standing.

For a competition prototype this is normal and fine. For a company it is company-ending — an acquirer's diligence or an enterprise customer's security review would surface it in an afternoon. The consequence is one of our deepest architecture principles (Chapter 7): **model sovereignty** — the core pipeline must never depend on a component we cannot legally use, and must treat every model as replaceable behind a stable interface. The near-term replacement candidates (commercially usable ASR, IndicTrans2 for translation, permissively licensed TTS and lip-sync models) are catalogued in Chapter 15.

1.4 What does not exist yet

An inventory, so nobody discovers these gaps mid-commitment. Grouped by the part of the handbook that addresses them:

- **Trust & legal** (Ch. 5, 12): consent capture, watermarking, content provenance, misuse policy, terms of service, data-protection posture (DPDP/GDPR), our own repo license.
- **Engineering platform** (Ch. 8, 10, 11, 13): auth, multi-tenancy, job queue, object storage, containerization, CI/CD, tests, observability beyond one append-only log file, staging, incident process.
- **AI research** (Ch. 15–20): golden evaluation sets, automatic dub-quality scoring, isochrony control, diarization/multi-speaker, emotion preservation, model registry, benchmark harness, A/B infrastructure.
- **Product** (Ch. 21–24): human-in-the-loop editor, API keys and metering, billing, pricing, SLAs, onboarding, accessibility, brand.
- **Business** (Ch. 25–26): named target customer, named competitors, GTM plan, unit economics, hiring plan, open-source decision, fundraising narrative.

The point of this list is sequencing, not shame. A company that builds all of this before finding a paying customer dies of thoroughness; one that finds customers without ever building it dies of success. The decision frameworks in Chapter 3 and the trigger rules in Chapter 14 exist to walk that line.

1.5 The honest performance baseline

From the competition guide and observed behavior: a 1-minute video takes roughly **3–5 minutes on CPU**; GPU brings it near **1.5× real time**. Whisper medium alone is a 1.5 GB download and the dominant memory consumer; Wav2Lip inference is the dominant GPU consumer. Nobody has yet measured **cost per dubbed minute** — the number that Chapter 11 establishes as the governing metric of the entire infrastructure, and Chapter 24 as the floor under every price we quote.

Quality has never been measured either. We have judge-impressions ("the sync is frame-perfect") but no word-error rate on our own test set, no translation adequacy score, no lip-sync error metric (LSE-C/LSE-D — Wav2Lip's own evaluation harness sits unused in `backend/Wav2Lip_repo/evaluation/`), and no human MOS panel. Chapter 16 exists because of this paragraph.

Key takeaways

- A real end-to-end Hindi→Marathi dubbing pipeline exists and works. That is rare and valuable.
- The code already carries good engineering instincts; the handbook's job is to systematize them, not import a foreign culture.
- The backend is single-tenant demo infrastructure and must not be marketed as more.
- **Every AI component in the pipeline is legally unusable for commerce.** Model sovereignty is therefore an architecture principle, not a preference.
- Nothing — cost, speed, or quality — is currently measured. Measurement precedes improvement, pricing, and model swaps alike.

Questions founders should ask

1. If a customer offered to pay tomorrow, which of the licensing items in §1.3 blocks taking their money, and what is the fastest legal path around each?
2. What is our actual cost per dubbed minute today, on CPU and on a rented GPU? (If unknown, that measurement is this week's work.)
3. Which single demo-grade item in §1.2 would embarrass us first with real users — and is it the same one we planned to fix first?
4. When this chapter is six months old, what in it will have silently become false?

Future research topics

- Benchmark the unused Wav2Lip evaluation harness (`evaluation/scores_LSE/`) on our own outputs to get a first LSE-C/LSE-D baseline before any model swap.
- Measure Whisper medium vs. small vs. large-v3 WER on a held-out Hindi education-video set; the current “medium” choice was never validated.
- Profile the pipeline stage-by-stage to find the true cost driver (hypothesis: Wav2Lip inference, but hypotheses are not measurements).

PART I — FOUNDER OS: HOW THE COMPANY THINKS

Chapter 2: Mission, Vision & the Category Thesis

Every accept/reject decision in this company traces back to this chapter. If a feature, hire, or contract cannot be justified in the vocabulary defined here, it is off-strategy no matter how attractive it looks.

2.1 Mission

Make every video understandable — and natural — in every Indian language.

Each word is chosen:

- **Every video**: lectures first, but the mission does not end at education.
- **Understandable *and natural***: subtitles make video understandable; SyncDub makes it *native* — the speaker's own voice, matched lips, preserved emotion. Naturalness is the product.
- **Every Indian language**: the wedge and, for the first several years, the boundary. We win Indic before we fight globally.

2.2 Vision (the ten-year claim)

In ten years, when an Indian institution — a university, a media house, a government ministry, a creator — produces video in one language, publishing it in ten languages is a checkbox, not a project. SyncDub is the infrastructure behind that checkbox: the quality standard, the API, and the editor that professionals reach for by default.

The honest version of this claim: dubbing at that scale will be a category with several winners globally. Our claim is narrower and therefore believable — **category leadership in Indic video localization**, on the strength of three things no global player will out-invest us in: Indic linguistic depth, an Indic correction-data flywheel (Chapter 20), and trust infrastructure suited to Indian institutions (Chapter 5).

2.3 The category thesis

Category: AI video localization — not "dubbing software," not "a Wav2Lip app." Localization includes translation quality, voice preservation, timing, lip sync, human review, and delivery. Companies that define themselves by one pipeline stage get commoditized by whoever owns the whole workflow.

Wedge: Indic languages, entering through education.

Why this wedge survives contact with competition:

1. **Structural neglect.** Global players (Rask, HeyGen, Papercup, Deepdub, ElevenLabs Dubbing, YouTube auto-dub) optimize for the revenue-dense European/East Asian

language pairs. Marathi prosody, Hindi-English code-switching, and Tamil isochrony are permanently third-priority for them. For us they are the whole company.

2. **Volume with tolerance.** Indian education content — coaching institutes, NPTEL-style university lectures, government skilling programs — is enormous in hours, chronically underfunded for human dubbing, and tolerant of very-good-but-not-cinematic quality. It is the ideal first customer: high volume to feed the data flywheel, forgiving enough to serve while quality matures.
3. **Price asymmetry.** Human dubbing costs orders of magnitude more per minute than our marginal cost even on today's unoptimized pipeline (Chapter 24). Global SaaS pricing, translated to INR, leaves a wide corridor we can occupy profitably.
4. **The speaker is the brand.** In education, students trust *the teacher*. Voice cloning that keeps the teacher's voice in Marathi is not a gimmick; it is the product's emotional core — and it is exactly what subtitle tools and generic-voice dubbing cannot offer.

Expansion order (each step earns the next): Hindi→Marathi education content → 4–5 major Indic languages, same vertical → creators and media houses → enterprise/government localization programs → non-Indic pairs only when Indic leadership is established. Chapter 25 details the go-to-market for each step.

2.4 What we are not

Negative space prevents drift. SyncDub is **not**:

- **A model company.** We do not compete with Whisper, IndicTrans2, or ElevenLabs at model research. We are the best *orchestrator and evaluator* of models for Indic localization (Chapter 15). If we ever train models, it is narrow fine-tuning fed by our correction data — never frontier research.
- **A subtitle tool.** Subtitles are a feature we may ship (the transcript already exists mid-pipeline); they are never the product.
- **A deepfake toolkit.** We re-voice and re-lip *consented* content. The consent boundary (Chapter 5) is part of the product definition, not a policy bolted on.
- **A services agency.** Human review is in the loop (Chapter 21), but we sell software and capacity, not per-project dubbing services. If early revenue arrives as services, it is tolerated only as a data-collection and learning channel with an explicit end date.

2.5 Strategy in one paragraph

Win Hindi→Marathi education dubbing so thoroughly that our quality metrics, our correction dataset, and our editor become the reference for Indic localization. Convert that position into an API and an enterprise product. Fund the widening of languages and verticals from revenue, not from hope. Let global players fight over Europe; by the time Indic matters to them, the flywheel (Chapter 4) should make the position expensive to attack.

Key takeaways

- Mission: make every video understandable and natural in every Indian language. Naturalness — the speaker's own voice and face — is the differentiator, not translation.
- The category is video *localization* (the whole workflow), the wedge is Indic education, and the ten-year claim is Indic category leadership, not global domination.
- The wedge works because of structural neglect by global players, huge tolerant volume, price asymmetry, and the teacher's-voice emotional core.
- We are not a model company, a subtitle tool, a deepfake toolkit, or an agency — and each "not" will be tested by a tempting opportunity within two years.

Questions founders should ask

1. Does this quarter's roadmap contain anything that serves "every language pair someday" at the expense of "Hindi→Marathi excellently now"?
2. If YouTube's auto-dubbing added Marathi tomorrow at zero cost, which paragraph of this chapter is our answer — and is it still true?
3. What evidence would falsify the education wedge (e.g., three coaching institutes that trial and churn), and are we honest enough to notice it?
4. Which "we are not" boundary is currently under the most commercial pressure?

Future research topics

- Size the Hindi→Marathi education market bottom-up: hours of content produced per year by the top 50 coaching institutes and ed-tech platforms.
- Catalogue where each global competitor actually stands on Indic output quality (run their trials; score them with our own Chapter 16 metrics — dual use: competitive intel and metric validation).
- Study Hindi-English code-switching frequency in real lecture content; it likely breaks both ASR and MT assumptions and could become an early technical differentiator.

Chapter 3: Core Values & Decision Frameworks

Values that don't decide anything are posters. Every value here is written as a tiebreaker — it tells you what to do when two good things conflict. The frameworks below are the company's default decision paths; deviating from them is allowed, but requires writing down why (see §3.4).

3.1 The decision stack

When priorities collide, they resolve in this order:

Trust > Quality > Cost > Speed.

1. **Trust** — consent, legality, provenance, data protection. Never traded away. A feature that grows revenue but breaches the Chapter 5 charter or ships an unlicensed model is rejected without a meeting. Trust ranks first not for moral display but because it is irreversible: quality can be improved next release; a consent scandal or a license lawsuit cannot be patched.
2. **Quality** — measured dub quality (Chapter 16), not aesthetic opinion. Outranks cost because in localization, quality is retention: a customer who ships one embarrassing dub to their students churns forever.
3. **Cost** — cost per dubbed minute (Chapter 11). Outranks speed because our margin *is* our strategy: the price corridor in Chapter 2 only exists if cost stays low.
4. **Speed** — of shipping and of processing. Last, but real: it breaks ties, and chronic slowness eventually becomes a quality and cost problem.

The stack governs trade-offs, not effort allocation. Most weeks are spent on speed and cost; that's fine — the stack only activates when they *conflict* with quality or trust.

3.2 Core values as tiebreakers

Measured beats impressive. Between a change that demos better and a change that scores better on the eval suite, the score wins. Corollary: work that creates measurement (a new metric, a golden set) is first-class engineering, not overhead. This value exists because our origin is a competition demo — impressiveness is our native talent and therefore our native bias.

Ship the ugly bridge. `clone_bridge.py` — a subprocess hop into a second Python environment because dependencies conflicted — is our house style: solve the problem crudely and *contained*, rather than beautifully and late, or not at all. The discipline that makes this safe is containment (the ugliness stays behind an interface) plus an explicit debt entry (Chapter 14). Ugly and spreading is not this value; it's the failure mode of it.

Challenge is a duty, not a right. Anyone — newest engineer included — is obligated to say "this is wrong, here is why, here is better." The brief that founded this company demanded that its own AI challenge it; employees get no less. The corollary duty: challenges come with reasons and alternatives, not vetoes.

The customer's student is the user. Our buyer is an institute; our real user is a student watching a dubbed lecture at 1.5× on a cheap phone. Decisions that please the buyer but degrade that student's experience (louder watermarks, quality-crushing compression tiers) get extra scrutiny.

Write it down. Decisions exist when they are written (§3.4). Culture, at 5 people or 500, is the sum of written decisions plus their enforcement.

3.3 The feature gauntlet

Every feature proposal answers the nine questions from the founding brief, plus one. Not all must be "yes" — but a proposal with fewer than three yeses, or a "no" on question 10, is rejected by default.

1. Does it create or deepen a competitive moat? (Chapter 4 defines which moats count.)
2. Does it increase customer retention?
3. Does it improve *measured* product quality?
4. Does it reduce cost per dubbed minute?
5. Does it improve scalability?
6. Does it create network or flywheel effects (especially: does it generate correction data)?
7. Does it improve enterprise adoption?
8. Does it improve developer adoption?
9. Does it make competitors' positions weaker?
10. **Can we ship it without violating the decision stack?** (Trust and licensing check — a hard gate, not a score.)

Two worked examples, using features actually latent in today's codebase:

- **"Export subtitles as SRT."** Moat: no. Retention: mild yes. Quality: no. Cost: no. Scalability: neutral. Flywheel: *yes* — subtitle correction is transcript correction, feeding the Chapter 20 flywheel. Enterprise: yes (accessibility requirements). Developer: mild yes. Competitors: no. Trust gate: pass. → **Accept**, cheaply: the transcript already exists mid-pipeline; this is exposure, not construction.
- **"Real-time dubbing of live streams."** Moat: maybe someday. Retention: unclear. Quality: would *reduce* it (no human review possible, isochrony unsolved — Chapter 17). Cost: explodes. Scalability: hard. Enterprise: not our current buyer. Trust gate: pass but pointless. → **Reject** for now, with a written revisit-condition: "batch quality ≥ human-parity on our eval suite and a named customer with budget."

3.4 Decision records

Any decision that is expensive to reverse — architecture, model/vendor choice, pricing, hiring, license — gets a **decision record**: a one-page markdown file in `docs/decisions/`, numbered, stating context, options considered, the decision, and the revisit-condition. The

revisit-condition is the innovation that keeps records from fossilizing: every record names the observable event that reopens it (“if GPU cost per minute drops below X”, “if this vendor’s license changes”).

Rules:

- Deciding without a record is allowed only for reversible decisions. If you’re unsure whether it’s reversible, it isn’t.
- Records are short. A record that takes more than an hour to write is hiding a disagreement that should be resolved in conversation first.
- Disagreement is recorded, then committed to. “Disagree and commit” only works if the disagreement is written down — that’s what makes the eventual “I told you so” productive instead of political.

3.5 How priorities are chosen (the quarterly loop)

1. Re-read Chapter 2. Anything on the candidate list that doesn’t serve the current wedge is deferred by default.
2. Score candidates through the gauntlet (§3.3).
3. Apply the **one-big-thing rule**: each quarter has exactly one headline objective (e.g., “replace the licensing-blocked pipeline stages”), because a team our size doing two big things does zero.
4. Reserve ~20% capacity for debt triggers (Chapter 14) and customer-driven interrupts. Planning to 100% is planning to slip.
5. Write the quarter as a decision record, revisit-conditions included.

Key takeaways

- The decision stack — Trust > Quality > Cost > Speed — resolves conflicts; it is ordered by irreversibility.
- Values are tiebreakers: measured beats impressive; ship the ugly bridge (contained, with a debt entry); challenge is a duty; the student is the user; write it down.
- Features pass a ten-question gauntlet; fewer than three yeses or a trust-gate failure means rejection by default.
- Irreversible decisions get one-page records with explicit revisit-conditions.
- One big thing per quarter; 20% capacity unplanned.

Questions founders should ask

1. What was the last decision made against the stack’s order — speed over quality, quality over trust — and was it written down or rationalized?
2. Which currently planned feature would fail the gauntlet if scored honestly today?

3. Are there decision records whose revisit-conditions have fired without anyone noticing?
4. Who challenged you last month, and what happened to them?

Future research topics

- Lightweight tooling: a `docs/decisions/` template plus a CI check that new model/vendor dependencies reference a decision record.
- Retrospective: score the three biggest past decisions (Whisper medium, XTTS bridge, Wav2Lip) through the gauntlet to calibrate how the framework would have judged them.

Chapter 4: The Moat Map

“Moat” is the most abused word in startup vocabulary, so this chapter is ruthless about it. A moat is something that makes each additional year of our existence *harder for a competitor to erase*. Most things that feel like moats aren’t. This chapter names the four we can actually build, the fake ones we must not fund as if they were real, and how each real moat compounds.

4.1 The test for a real moat

Ask of any claimed moat: **“If a well-funded team cloned our product tonight, what would they still not have in a year?”** Anything they’d have by then — features, UI, model wiring — is not a moat. It may still be worth building; it’s just not defensible, and it must not be *funded or prioritized as if it were*.

4.2 Moat #1: The correction-data flywheel (primary)

Every professional dub gets human review (Chapter 21). Every correction an editor makes — a mistranslated idiom fixed, a segment re-timed, a wrong voice-gender overridden, an emphasis restored — is a labeled example of *exactly where automated Indic dubbing fails*. Nobody can buy this dataset, because it doesn’t exist anywhere: it is generated only by running real Indic content through a real pipeline in front of real editors.

The flywheel: more customers → more corrections → better models and better automatic QA (Chapter 20) → higher quality → more customers. Each loop is small; the compounding is the moat. The cloning team in §4.1 starts this flywheel at zero regardless of their funding.

Design consequence, stated early because everything depends on it: **the editor is not a UI feature; it is the data engine.** Corrections must be captured as structured data (what changed, at which pipeline stage, on what input) from the very first version — retrofitting structure onto freetext edits later is archaeology.

4.3 Moat #2: Quality-metric ownership

Whoever defines how Indic dub quality is *measured* owns the category’s terms of trade. Today nobody measures it — including us (Chapter 1, §1.5). Chapter 16 builds the evaluation OS; this section states the strategic intent behind it:

- Internally, the metric suite is what makes model swapping possible — you cannot replace Wav2Lip or XTTS (which licensing forces us to do) without a score that says the replacement is no worse.
- Externally, published benchmarks — “here is the standard Hindi→Marathi dubbing test set, here is how every tool scores” — position us as the category’s referee. Referees are hard to displace, and a benchmark we publish is a benchmark tuned to what we’re best at: Indic.

The metric moat feeds the data moat: automatic quality scoring decides *which* outputs need human review, making editors more efficient, making corrections cheaper, spinning the flywheel faster.

4.4 Moat #3: Indic depth

Hindi-English code-switching mid-sentence. Marathi honorifics that change verb forms. Devanagari numerals in one language, spoken digits in another. Regional accent robustness. Isochrony patterns specific to Indic syllable timing (Chapter 17). Each is a small, unglamorous problem; a hundred of them solved is a capability global players cannot fast-follow, because each fix came from a correction in the flywheel and lives in our test sets.

Indic depth is a *derived* moat — it is what moats #1 and #2 precipitate when pointed at Indic content — but it deserves its own name because it is the moat customers actually *feel*: our Marathi sounds right and theirs doesn't.

4.5 Moat #4: Trust infrastructure

Consent verification, watermarking, provenance signing, data-protection posture (Chapter 5). For creators this is table stakes; for our real expansion targets — education boards, media houses, government programs — it is the *procurement gate*. A competitor with better lip sync but no consent chain loses the government deal to us. Trust compounds slowly (audits passed, years without incident) and transfers poorly (a competitor cannot copy our compliance history), which is what makes it a moat rather than a checkbox.

4.6 Fake moats — name them so we don't fund them

- **Model wiring.** Today's pipeline — Whisper + translator + TTS + Wav2Lip — can be reassembled by a competent team in weeks (we know; that's roughly how it was built). Integration is table stakes.
- **Any specific model.** Models are depreciating assets we don't even own (Chapter 1 §1.3). Betting the company's defensibility on privileged access to a model is betting on someone else's roadmap.
- **UI polish.** Copyable in a design sprint. The editor's *data capture* is a moat; its pixels are not.
- **Feature count.** The gauntlet (Chapter 3 §3.3) exists precisely because feature breadth feels like progress while diluting the moats that matter.
- **First-mover status.** Being early in Indic dubbing earns us a head start on the flywheel — nothing more. The head start is only worth what we convert it into.

Fake moats may still be *worth building* (we obviously need a UI and wired models). The rule is about investment framing: fund them as costs of doing business, sized accordingly — never as differentiation.

4.7 Sequencing the moats

Moats compound, so start order matters more than effort split:

1. **Now (pre-revenue):** metric ownership starts immediately — golden set and baseline scores (Chapter 16) cost weeks and unblock everything else, including the licensing-forced model swaps. Trust basics (consent capture, our own LICENSE, a misuse policy) are days of work with outsized enterprise payoff later.
2. **First customers:** the editor ships with structured correction capture from day one — even if the editor is crude, the *data schema* must be right.
3. **Growth:** flywheel volume; publish the benchmark; let Indic depth accumulate in test sets rather than tribal knowledge.

Key takeaways

- The moat test: what would a well-funded clone still lack after a year? Fund only that as differentiation.
- Four real moats: correction-data flywheel (primary), quality-metric ownership, Indic depth, trust infrastructure. The first two generate the third; the fourth gates enterprise.
- The editor is the data engine — structured correction capture is non-negotiable from v1.
- Model wiring, specific models, UI polish, feature count, and first-mover status are fake moats: build as needed, never fund as defensibility.

Questions founders should ask

1. What fraction of this quarter’s engineering effort deepens a real moat, honestly tallied?
2. If we lost access to every third-party model tomorrow, which of our assets retain value? (Answer should be: the data, the metrics, the customers, the trust record.)
3. Is correction data actually accumulating in a schema a future fine-tuning run could consume — or as unstructured edits?
4. Which competitor is closest to publishing an Indic dubbing benchmark before we do?

Future research topics

- Design the correction-data schema now (per-stage, per-segment, before/after, editor rationale) even though the editor doesn’t exist yet — it constrains the editor’s design, not vice versa.
- Investigate precedents of metric-ownership strategies (e.g., benchmark suites that shaped ML subfields) for what made them stick.

- Estimate flywheel economics: corrections per dubbed hour, editor cost per correction, and the quality lift per thousand corrections — the three numbers that decide how fast moat #1 actually spins.

Chapter 5: The Responsible Synthetic Media Charter

SyncDub clones voices and re-animates faces. That is deepfake-adjacent technology, and pretending otherwise is how companies in this category end up as cautionary headlines. This charter is the trust layer of the decision stack (Chapter 3): it is never traded away, and every employee is empowered to block a release that violates it. It is also — deliberately — a sales weapon: Chapter 4 established trust infrastructure as a moat, and Chapter 23 shows it winning procurement. Ethics and strategy point the same direction here; that alignment is why this charter will actually be followed.

5.1 The bright line

SyncDub transforms consented content for its rightful owner. It does not put words in anyone's mouth.

Everything we build re-expresses what a speaker *actually said*, in another language, with their consent. The moment a tool of ours can make a person appear to say something they never said, we have left our product category and entered another one — one we have chosen not to be in (Chapter 2, §2.4).

Concrete consequences of the bright line:

- No "type text, get the speaker saying it" feature, ever, regardless of revenue attached. The TTS stage is only ever fed by the translation of the speaker's own transcript. (Editor corrections to translations are bounded rephrasings of the source — audited, attributed, and logged, per §5.4.)
- No dubbing of content the uploader does not have rights to. "It's for education" is not a rights claim.
- No voice cloning without speaker-level consent (§5.2) — the uploader owning the *video* does not imply the speaker consented to *voice cloning*.

5.2 Consent, in layers

Consent is a chain, and today's codebase has none of it — any video uploaded to `backend/server.py` is processed, no questions asked. The charter requires three layers, phased in as the product matures (Chapter 8 sequences the implementation):

1. **Uploader attestation (v1, cheap, immediate):** at upload, the customer attests they hold the rights to the content and the authority to localize it. Recorded, timestamped, tied to the account. This is a legal instrument, not a UX hurdle — one checkbox with real words.
2. **Speaker consent for voice cloning (v1 for cloning path):** cloning (`USE_VOICE_CLONING = True` today, silently) becomes opt-in per project, with a recorded attestation that identified speakers consented to voice replication. Default path remains stock neural voices.

3. **Verified consent (enterprise):** for institutional deals, speaker consent collected verifiably — signed release or an in-product spoken-consent flow ("I authorize SyncDub to replicate my voice for localization of my content", in the speaker's own cloned-reference voice sample). This is the procurement-gate version.

5.3 Provenance and watermarking

Every SyncDub output must be *identifiable as synthetic* by machines, and *attributable* by us:

- **Audible/visible disclosure** where the customer's jurisdiction or platform requires it; controllable by the customer otherwise — but the metadata layer below is never customer-controllable.
- **Content credentials (C2PA)**: sign outputs with provenance metadata — produced by SyncDub, from which source asset, for which account — as soon as engineering capacity allows (this is a known, standardized integration, not research).
- **Forensic watermarking** (inaudible/invisible): the aspiration is real but the technology is an arms race; we adopt the best available standard rather than inventing one, and we never *claim* robustness we don't have.
- **Internal attribution ledger (v1, immediate):** even before C2PA, we keep hashes of every input and output tied to the producing account and job. When a video surfaces and someone asks "did SyncDub make this?", we must be able to answer. Today we cannot — outputs aren't even uniquely named (`final_dubbed.mp4`, Chapter 1 §1.2).

5.4 Auditability

Every job retains, for a defined period: who submitted it, the consent attestations, the input hash, the transcript, the translation (with any editor modifications attributed to the editing account), and the output hash. This audit trail is what turns the bright line from a promise into a checkable fact — and it doubles as the data layer the flywheel needs anyway (Chapter 20). Trust and moat, one schema.

5.5 Misuse policy and enforcement

- **Refusal categories:** impersonation of public figures without documented authority; political campaign material (categorically, in v1 — the review capacity to do it safely doesn't exist); content the uploader cannot claim rights to; sexual content involving real persons.
- **Detection posture:** we are realistic — a determined bad actor with a consented-looking upload can fool us. The policy is therefore built on attestation + attribution + takedown: fraud is the *uploader's* documented lie, we can prove what we made, and we can act fast when notified.
- **Takedown:** a public reporting channel; verified reports get the output's distribution support revoked, the account reviewed, and law enforcement cooperation where

warranted. Response-time targets live in the support standards (Chapter 23).

- **Enforcement inward:** any employee can halt a feature or a deal on charter grounds by writing down the objection (Chapter 3's challenge duty). The objection is resolved by the founder in writing. A charter that can be waived verbally under quarterly pressure is not a charter.

5.6 Regulatory posture

We build *ahead* of Indian regulation, not behind it: India's DPDP Act (personal data — and a voice print is personal data), IT Rules on synthetic media disclosure as they evolve, and the EU AI Act's transparency obligations as the likely global template. The stance in one line: **compliance is a product feature we ship early, because our buyers' lawyers will eventually require it and our competitors will retrofit it in panic.** Chapter 12 carries the engineering specifics (retention, deletion, data residency).

Key takeaways

- The bright line: consented transformation of real content, never fabrication. No text-to-speaker feature, ever.
- Consent is layered — uploader attestation now, speaker consent for cloning now, verified consent for enterprise — and today's code has none of it.
- Every output must be attributable (hash ledger now, C2PA soon); every job auditable end-to-end.
- Misuse policy = attestation + attribution + fast takedown, enforced by anyone in the company, in writing.
- Trust is simultaneously an ethical floor and a competitive moat; that alignment is what makes it durable.

Questions founders should ask

1. Could we, today, prove whether a given viral video was produced by our pipeline? (Today: no. When does that become yes?)
2. Which charter commitment would be most tempting to waive for our first large deal, and is the enforcement path (§5.5) strong enough to survive that day?
3. Is the cloning path (`USE_VOICE_CLONING`) still on-by-default without consent capture? Why?
4. Whose voice-consent gets violated *first* in our wedge market — the professor whose old lectures the institute uploads? What does our attestation actually say about that case?

Future research topics

- C2PA integration effort estimate for our MoviePy/ffmpeg render stage.

- Survey audio watermarking robustness (re-encoding, platform transcoding) to calibrate what we can honestly claim.
- Track DPDP rules and IT-Rules amendments on synthetic media; assign an owner who reads the actual notifications, not the news coverage.

Chapter 6: From One Founder to 500 Engineers

The founding brief imagined a handbook for 500 engineers. The only org that matters right now is one founder deciding who hires #1 through #5 are. This chapter serves Monday morning first and the 500-engineer company second — with the scaling principles stated so the early decisions don't have to be unwound later.

6.1 The founder's actual job, by stage

- **Now (team of one):** the founder is the bottleneck on everything; the job is choosing which bottleneck to *be*. Per the moat map, the founder's irreplaceable work is customer discovery in the wedge market and the quality bar — everything else (pipeline hardening, UI) is delegable to the first hires. The trap at this stage is engineering comfort: rebuilding the backend feels like progress and postpones the phone calls that decide whether the company should exist.
- **1→5:** the founder still reviews everything; the handbook is enforced by proximity.
- **5→25:** the handbook replaces proximity. Decision records (Chapter 3 §3.4) and review checklists (Appendix A) stop being founder tools and become the management system.
- **25→500:** the founder's job is protecting the decision stack and the charter against scale's default drift (speed over quality, growth over trust). Everything else has an owner who isn't the founder.

6.2 The first five hires

Sequenced against the moats and the roadmap, not against org-chart convention:

1. **#1 — Pipeline/ML engineer.** Owns the licensing-forced model swaps (Chapter 15) and the evaluation OS (Chapter 16). The single highest-leverage hire: they unblock both legality and measurability. Profile: has shipped ML systems to production, allergic to unmeasured claims, comfortable in the ugly-bridge house style.
2. **#2 — Product/full-stack engineer.** Owns demo-to-service (Chapter 8) and then the editor (Chapter 21) with its correction-data capture. Profile: has built multi-tenant SaaS before; treats data schemas as product decisions.
3. **#3 — Founding editor/linguist (part-time is fine).** A working Marathi translator/dubbing professional. Not an engineer — the person who makes our quality metrics mean something and our golden sets honest. Cheap, rare thinking: every dubbing-tech company discovers too late that linguists should have been in the room from the start.
4. **#4 — Infra/SRE-minded engineer.** Owns GPU cost (Chapter 11), reliability (Chapter 13), security hardening (Chapter 12). Until this hire, #1 and #2 share the pager.
5. **#5 — Growth/customer engineer.** Sits with wedge customers (coaching institutes), owns onboarding and the feedback loop into the roadmap. Half sales-engineer, half

support; the founder's successor on customer discovery so the founder can raise/expand.

Non-hires at this stage, stated to resist pressure: no designers-as-employees (contract it), no "Head of AI" (hire #1 *is* AI), no salespeople before the product can be sold without the founder in the room (Chapter 25), no managers of any kind.

6.3 Hiring philosophy

- **Hire for the eval, not the vibe.** Every candidate does a paid work-sample shaped like the real job: #1 candidates benchmark two ASR models on a Hindi test set and write up the trade-off; #2 candidates design the job-queue migration for `backend/server.py`. Their write-up is a decision record; judge it like one. Interviews measure interviewing; work samples measure work.
- **Values screen = the challenge test.** During the work sample, we assert something wrong. Candidates who push back with reasons pass the culture bar (Chapter 3's challenge duty); candidates who fold — however brilliant — fail it. At five people, one agreeable genius quietly breaks the decision stack.
- **Indic context is a real qualification.** A Marathi-speaking engineer hears our failure modes; treat language skill as the hiring signal it is, not a nice-to-have.
- **Pay honestly at the bottom of the market's top.** We won't outbid big tech; we offer real equity, real ownership of a chapter of this handbook, and a mission with a face (the student in Chapter 3's values). Candidates who need the highest number should take it elsewhere without hard feelings.
- **Fire fast on trust, slow on skill.** A charter violation or a faked result ends employment immediately. A skill gap gets a written expectation, real help, and one honest quarter.

6.4 How the org scales without becoming an org chart

Principles that hold from 5 to 500:

- **Owners, not committees.** Every system, metric, and chapter of this handbook has exactly one named owner. Shared ownership is unowned.
- **The handbook is the manager until managers exist.** First management layer arrives around 12–15 people, and managers are working engineers/editors first.
- **Teams form around moats, not technologies.** Future teams are "evaluation," "editor/flywheel," "trust," "cost" — not "frontend team" and "backend team." Technology-shaped teams optimize their layer; moat-shaped teams optimize the company.
- **Every engineer touches customer content monthly.** Watching a real dubbed lecture with real corrections is the antidote to building for the demo again.

Key takeaways

- The founder’s scarce work is customer discovery and the quality bar; the engineering itch is the trap.
- Five hires, in order: ML/eval engineer, product/SaaS engineer, founding linguist, infra/SRE, growth/customer engineer — sequenced by moat, with the linguist as the unconventional early call.
- Hire by paid work samples judged as decision records; screen values by testing the challenge duty; treat Marathi/Indic fluency as a technical qualification.
- Scale by named owners and moat-shaped teams; the handbook replaces proximity before managers replace the handbook.

Questions founders should ask

1. This month, how many hours went to customer discovery versus code? Is that ratio a decision or a drift?
2. If hire #1 started Monday, is there a written work-sample brief and a golden-set seed for them — or would their first week be spent watching you work?
3. Who is the named owner of each shipped system today? (Every “me, I guess” is a pending hire or a pending handoff.)
4. Which handbook chapter would you trust the least in a new hire’s hands, and what does that say about the chapter?

Future research topics

- Draft the two work-sample briefs (#1 and #2) now — they’re useful even before hiring, as scoping documents for the actual work.
- Compensation benchmarks for ML engineers in Indian startup market vs. equity expectations; decide the equity budget for hires #1–5 as one decision record, not five negotiations.
- Talk to founders of Indic-language ML companies (AI4Bharat orbit, Sarvam-adjacent) about where linguist-in-the-loop hiring worked or failed.

PART II — ENGINEERING OS: HOW ENGINEERS THINK

Chapter 7: Architecture Principles

These principles govern every architecture decision at SyncDub. They are few, they are ranked, and each one earns its place by referencing a real scar or a real moat — not by being a best practice somewhere else.

7.1 Principle 1: Model sovereignty

The core pipeline must never depend on a component we cannot legally use, and every model must be replaceable behind a stable interface.

This is the Wav2Lip lesson (Chapter 1 §1.3) made permanent: our lip-sync stage is owned by a competitor, our voice cloning is non-commercial-only with no counterparty to license from, and our translation rides an unofficial scraper. Sovereignty has two halves:

- **Legal sovereignty:** every model, weight file, and API in the serving path has a license or contract that permits commercial use, recorded in a decision record (Chapter 3 §3.4) with its revisit-condition ("if this license changes…", "if this vendor is acquired by a competitor…"). A CI check should eventually refuse builds that introduce dependencies without a license record.
- **Technical sovereignty:** each pipeline stage (ASR, MT, TTS, lip sync, and later diarization) is consumed through a stage interface — `Transcriber`, `Translator`, `Synthesizer`, `LipSyncer` — that names *what the stage does*, not which model does it. Today `inference_marathi.py` calls Whisper, GoogleTranslator, and the XTTS bridge directly by name; the refactor to interfaces is priced into the demo-to-service migration (Chapter 8). The bar: swapping a stage's model must never touch code outside that stage's adapter, and must always be justified by evaluation scores (Chapter 16), never by vibes.

The founding brief said "never lock the company to today's AI models." This principle is how that sentence becomes enforceable.

7.2 Principle 2: The pipeline is jobs, not calls

Dubbing is minutes-long, GPU-hungry, multi-stage work. Architecturally it is a **job system** — durable jobs with IDs, states, retries, and per-stage artifacts — never a request/response call, and never `subprocess.Popen` fire-and-forget (today's reality in `backend/server.py`). Consequences:

- Every stage writes its artifact (audio, transcript, translation, synthesized audio, rendered video) to durable storage keyed by job ID. Stages become resumable and independently retryable; a Wav2Lip crash doesn't re-run Whisper.
- Job state is the single source of truth the frontend polls and the audit trail (Chapter 5 §5.4) reads — one schema serving product, trust, and the flywheel at once.

- Progress reporting is a property of the job record, not a global `progress.json` file.

Chapter 8 is the migration plan; this principle is why it isn't optional.

7.3 Principle 3: Contain the ugly

The house style (Chapter 3): crude solutions are welcome when they are *contained* — behind an interface, in their own process or environment, with a debt entry. `clone_bridge.py` is the canonical example: dependency hell resolved by a subprocess boundary instead of a three-week environment unification. The rule that keeps this healthy:

Ugliness may be deep, never wide. A hack inside one stage adapter is fine. A hack that forces callers to know about it (a magic filename, a shared global file, an env var like today's hardcoded `IMAGEIO_FFMPEG_EXE` path) has escaped containment and must be driven back in at the next touch.

7.4 Principle 4: Measured evolution

No architecture change ships on aesthetics. Every significant change carries: the metric it should move (cost per dubbed minute, p95 job latency, eval-suite quality score), the measurement before, and the measurement after. This is the engineering face of "measured beats impressive" (Chapter 3) — and it cuts both ways: it blocks resume-driven rewrites *and* it justifies genuinely needed ones with numbers instead of arguments.

Corollary — **boring by default**: Postgres before exotic stores, one queue before an event mesh, a monolith with clean stage interfaces before microservices. The founding brief asked for "Microservice Standards"; our standard is that microservices are an *outcome* of measured scaling pressure on a specific stage (GPU stages will likely split first — Chapter 11), never a starting shape. A five-person company running twelve services is doing distributed-systems cosplay.

7.5 Principle 5: The data schema is the architecture

Chapters 4 and 5 both land on the same artifact: the job/segment/correction schema. Per-segment records — source text, timing, translation, edits, quality scores — are simultaneously the moat's raw material, the audit trail, and the pipeline's working state. Design consequence: **schema changes get the most senior review of any change in the company**, and stage adapters may be rewritten freely while the schema evolves only additively. Code is replaceable; the accumulated data is the company.

7.6 Anti-principles

Things we explicitly do not value architecturally: horizontal scale before there is load; multi-cloud abstraction before there is one cloud bill worth optimizing; plugin systems before there are two real consumers; configurability before there are two real configurations; real-time paths before batch quality is won (Chapter 3's worked rejection).

Each of these is a form of pretending to be the 500-engineer company instead of becoming it.

Key takeaways

- Model sovereignty — legal and technical — is principle #1, born from the fact that today zero pipeline components are commercially usable.
- The pipeline is a durable job system with per-stage artifacts; request/response and fire-and-forget are architectural bugs.
- Ugly is fine when deep and contained; ugly that callers can see has escaped and must be recaptured.
- Architecture changes ship with before/after measurements; boring technology is the default; microservices are an outcome, not a plan.
- The job/segment/correction schema outranks all code — it is moat, audit trail, and state in one, and it evolves additively under senior review.

Questions founders should ask

1. Which pipeline stages are still consumed as direct imports rather than through a stage interface, and what does each swap cost today versus after Chapter 8?
2. Does every model in the serving path have a license decision record yet?
3. What was the last architecture change shipped without a before/after metric — and did it actually help?
4. Where has ugliness gone wide (visible to callers) rather than deep this quarter?

Future research topics

- Define the v1 stage-interface signatures (`Transcriber`, `Translator`, `Synthesizer`, `LipSyncer`) against the current call sites in `inference_marathi.py` — the cheapest possible start on sovereignty.
- Draft the v1 job/segment schema jointly with the correction-schema work from Chapter 4’s research topics; they must be one design exercise, not two.
- Evaluate queue options at our scale (a Postgres-backed queue is likely sufficient for years) with a one-page decision record.

Chapter 8: From Demo to Service

This is the next engineering epic, written as a concrete migration plan from the code that exists (`backend/server.py`, `backend/inference_marathi.py`) to a service that can take money. It is deliberately staged so that each stage ships value alone — the migration can pause at any stage without leaving the system worse than it started.

8.1 Where we are (recap of the gap)

One FastAPI process; upload spawns `inference_marathi.py` via `subprocess.Popen` fire-and-forget; a single global `progress.json`; output always `outputs/final_dubbed.mp4`; no auth; CORS `*`; uploaded filename used as the disk path; Windows-hardcoded ffmpeg location. Single-tenant by construction (Chapter 1 §1.2).

8.2 Target shape (v1 service, not v10 platform)

A deliberately boring target, per Chapter 7's principles:

- **One API service** (FastAPI stays) + **one worker process** consuming a **Postgres-backed job queue** + **object storage** for artifacts. No Kubernetes, no Redis, no microservices. Postgres does jobs, users, segments, and audit in one place.
- **Job model:** `job(id, account_id, state, created_at, ...)`, states `queued → running(stage) → review → done | failed(stage, error)`; per-stage artifact rows pointing into object storage; per-segment rows carrying transcript/translation/timing (the Chapter 7 §7.5 schema).
- **Stage interfaces:** `Transcriber / Translator / Synthesizer / LipSyncer` adapters wrapping today's exact implementations — the sovereignty refactor (Chapter 7 §7.1) done as part of the move, not as a separate rewrite.
- **Auth:** account-scoped API keys (hashed at rest). Enough for first customers and the API product's foundation (Chapter 22); OAuth/SSO waits for enterprise pull (Chapter 23).
- **Consent capture:** the uploader attestation and cloning opt-in from Chapter 5 §5.2 enter the upload flow *here*, in v1 — retrofitting consent after customers exist is far harder.

8.3 The staged migration

Each stage is independently shippable and reversible:

1. **Stage 0 — Portability (days).** Kill the hardcoded `D:/Miniconda3/...` ffmpeg path and `COQUI_TOS_AGREED` scattering; configuration via environment/settings file; a `Dockerfile` that runs the pipeline end-to-end on CPU. Exit test: a teammate (or CI) runs the pipeline on Linux from a fresh clone.

2. **Stage 1 — Job identity (days).** Every upload gets a UUID; inputs stored as `uploads/{job_id}/source.mp4` (killing the filename-as-path traversal risk); outputs as `outputs/{job_id}/dubbed.mp4`; `progress.json` becomes `progress/{job_id}.json`. Frontend passes the job ID. Two users stop corrupting each other *before* any queue exists.
3. **Stage 2 — The queue (week).** Jobs table in Postgres; API enqueues; a worker loop claims jobs (`SELECT ... FOR UPDATE SKIP LOCKED`), runs the pipeline in-process, heartbeats, and writes state. `Popen` dies here. Retry-per-stage arrives with per-stage artifact records. Concurrency = number of workers, controlled and observable.
4. **Stage 3 — Accounts, keys, consent (week).** Accounts table, API keys, per-account job listing; the attestation checkbox and cloning opt-in recorded on the job row; CORS locked to the real frontend origin. The audit ledger (input/output hashes, Chapter 5 §5.3) is two columns on tables that now exist.
5. **Stage 4 — Object storage & artifacts (week).** Local disk paths replaced by S3-compatible storage keyed by job; signed URLs replace the static `/outputs` mount; retention policy becomes a deletable prefix per job (the Chapter 12 deletion story starts working).
6. **Stage 5 — Observability (ongoing).** Structured logs with `job_id`/`stage` fields replacing the single `pipeline.log`; per-stage duration and cost metrics emitted — feeding the cost-per-dubbed-minute measurement Chapter 11 governs by.

Sequencing rationale: identity before queue (Stage 1's UUID work is what Stage 2's schema keys on), queue before accounts (accounts without job isolation are cosmetic), storage after accounts (signed URLs need identity). The consent items ride Stage 3 because that's when the schema they attach to exists.

8.4 What we keep

Migration plans fail by rewriting what works. Explicitly preserved: the pipeline logic itself (`inference_marathi.py`'s stage functions move behind adapters largely intact), the lazy load/unload memory discipline, the atomic-write pattern (now applied to DB transactions instead of JSON files), the `clone_bridge.py` subprocess isolation (now invoked by the worker), and the React frontend (its polling model maps directly onto job-state polling).

8.5 What "done" means

The migration is complete when: two customers can run jobs concurrently without interference; a killed worker resumes or retries its job without operator archaeology; every output traces to an account, an attestation, and input/output hashes; and a new engineer can run the whole system locally with `docker compose up`. That last clause is also the hiring work-sample environment (Chapter 6 §6.3).

Common mistakes this plan is designed against

- **The platform rewrite.** Kubernetes + microservices + Redis + a message bus for a system with zero concurrent users. Our ceiling-check: Postgres-backed queues comfortably serve thousands of jobs/day; we will be thrilled to have that problem.
- **The parallel system.** Building v2 beside v1 and cutting over "when ready." Every stage above modifies the live system and ships alone.
- **Auth last.** Deferring accounts until "after launch" means retrofitting tenancy onto data with no tenant column. Stage 3 is early on purpose.
- **Skipping Stage 0.** Every subsequent stage is untestable by anyone but the founder's Windows machine until portability lands.

Key takeaways

- Target: API + Postgres queue + worker + object storage. Boring, sufficient for years, and each piece earns later replacements with measurements (Chapter 7 §7.4).
- Six stages, each shippable alone: portability → job identity → queue → accounts/consent → object storage → observability.
- Consent capture and the audit ledger enter at Stage 3 — trust infrastructure rides the migration rather than following it.
- The pipeline's working logic, memory discipline, and bridge isolation are kept, not rewritten.

Questions founders should ask

1. Which stage is live today, and does anything in flight violate "each stage ships alone"?
2. Has anyone other than the founder run the full pipeline end-to-end yet? (Stage 0's exit test is the hiring unblock.)
3. Are consent attestations actually recorded on jobs, or did Stage 3 ship as auth-only under time pressure?
4. What did Stage 5's first cost-per-minute numbers reveal, and did they surprise us?

Future research topics

- Postgres queue implementation choice (hand-rolled `SKIP LOCKED` loop vs. a small library) — one-page decision record.
- Signed-URL expiry and download UX for large video files on Indian mobile networks.
- Whether Whisper/Wav2Lip model weights live in the image, a volume, or a model cache service at Stage 0 — affects both cold-start time and the Chapter 15 registry design.

Chapter 9: Code, Repository & API Standards

Standards for a five-person company, strict exactly where violations compound (interfaces, schemas, dependencies, public APIs) and deliberately loose where they don't (formatting taste, internal style debates). Every rule here is enforceable by a machine or a checklist — standards that rely on remembering are wishes.

9.1 Repository standards

- **One repo.** The monorepo (backend, frontend, docs, this handbook) stays until a measured reason splits it. Vendored third-party code lives under a `third_party/` path with its license file preserved — `backend/Wav2Lip_repo/` is grandfathered until Chapter 15's swap retires it, but no new vendoring without a license decision record (Chapter 7 §7.1).
- **A `LICENSE` file at root** — currently missing (Chapter 1 §1.2). Until the open-source decision (Chapter 26) is made, the repo is explicitly proprietary; an empty license field is not a neutral default, it is ambiguity.
- **Hygiene enforced by CI, not comments:** formatter (black/ruff for Python, prettier for JS) and linter run on every PR; a failing check blocks merge. No style debates in review — the formatter is the arbiter, chosen once in a decision record.
- **No artifacts in git:** model weights, videos, `progress.json`, logs are gitignored. Weights are fetched by the Stage-0 setup (Chapter 8), never committed.
- **Scratch code has a home:** `scratch/` is gitignored; the root-level `test.py`, `TP.py`, `filelist.txt` era ends. If an experiment is worth sharing, it becomes a documented script under `tools/`; if not, it stays out of history.

9.2 Code standards

- **The interface line is the quality line.** Inside a stage adapter, pragmatic code is fine (Chapter 7 §7.3). At interfaces — stage adapters, API handlers, schema definitions — full rigor: type hints, docstrings stating contract and failure modes, and tests (Chapter 10). This two-tier standard is written down precisely so reviews argue about *which tier code is in*, not about taste.
- **Errors are data.** Pipeline failures must set job state with stage and cause — today's pattern of `except: pass` (see the CUDA cleanup in `unload_models()`) and log-and-continue is acceptable only where the failure is genuinely ignorable, and that judgment goes in a comment. A silent failure in ASR shows up as a mystery in lip-sync; error context is how a two-person team debugs a five-stage pipeline.
- **Configuration is explicit.** One settings module, env-var driven, no `os.environ` writes scattered at import time (today: `IMAGEIO_FFMPEG_EXE` and `COQUI_TOS_AGREED` set as side effects at the top of `inference_marathi.py`).
- **Comments state constraints, not narration.** The good existing example: the note that `beam_size=5` was tried and wasn't worth it — that's a measurement preserved.

The standard: comments record *why* and *what was ruled out*, never what the next line does.

9.3 Code review standards

- **Every change is a PR; every PR has one reviewer.** At current team size the reviewer may be "future you after a night's sleep" for solo work — the discipline of writing the description ("what, why, how verified") is most of the value.
- **Review priorities, in order:** correctness of schema/interface changes → security and trust implications (Chapter 5, 12) → measurement claims ("makes it faster" needs a number) → containment (did ugliness leak wide?) → everything else. A reviewer's approval means "I understand this and would maintain it," not "I skimmed it."
- **Schema changes get the most senior review in the company** (Chapter 7 §7.5) and must be additive unless a decision record says otherwise.
- **Review SLA: one business day.** Blocked authors may escalate; stale reviews are the founder's problem to unblock, because review latency silently sets the company's shipping speed.

9.4 API standards

The API is a future product (Chapter 22); these standards start applying with Stage 2–3 of the migration (Chapter 8):

- **Versioned from the first public consumer:** `/v1/` prefix; breaking changes require a new version and a deprecation window, never an in-place change. Internal-only endpoints may break freely until a customer holds a key.
- **Resource-shaped, job-centric:** `POST /v1/jobs` (create dubbing job), `GET /v1/jobs/{id}` (state + artifact URLs), `GET /v1/jobs/{id}/segments` (transcript/translation for the editor). Verbs live in state transitions, not URL names.
- **Errors follow one envelope:** machine-readable code, human message, `job_id` / `stage` context where applicable. The frontend's current guess-from-shape parsing ends with this.
- **Idempotency for anything billable:** job creation accepts an idempotency key, because customers on Indian mobile networks *will* retry uploads, and double-charging a dub is a trust incident, not a bug.
- **Nothing undocumented is public.** An endpoint is public when it appears in the API reference with an example — not when it happens to be reachable.

9.5 Dependency standards

- New runtime dependencies require: a license check (the Chapter 1 lesson — a `requirements.txt` line is a legal decision), a maintenance sanity check (is it alive?), and a one-line justification in the PR.

- Pin everything: `requirements.txt` moves to pinned versions with a lock/constraints file; today's unpinned installs mean two clones of the repo can behave differently — which, combined with the Whisper/XTTS environment split, is how "works on my machine" becomes the whole engineering culture.
- The `clone_bridge.py` second environment is documented as what it is: a pinned, containerized sidecar after Stage 0, not folklore about "the other conda env."

Key takeaways

- Strict where violations compound (interfaces, schemas, deps, public API), loose where they don't; machines enforce, not memory.
- The repo gets a LICENSE, CI-enforced formatting, no committed artifacts, and an end to root-level scratch scripts.
- Two-tier code standard: pragmatic inside adapters, rigorous at interfaces; errors carry stage context; configuration is centralized.
- Reviews prioritize schema > trust > measurement claims > containment; schema changes get the most senior eyes.
- The API is versioned, job-centric, idempotent where money moves, and public only when documented.

Questions founders should ask

1. Is CI actually blocking merges yet, or are these standards still enforced by memory?
2. What was the last dependency added, and did anyone read its license before merging?
3. Which interface currently has the worst gap between its importance and its documentation?
4. Is review latency measured in hours or in "when someone remembers"?

Future research topics

- Set up the minimal CI (format, lint, and Chapter 10's tests) as part of Stage 0 — it is the cheapest cultural investment available.
- Evaluate a license-scanning step (e.g., pip-licenses in CI) to mechanize the §9.5 check.
- Define the `/v1/jobs` OpenAPI spec early — it doubles as the design document for Chapter 8's Stage 2–3.

Chapter 10: Testing & Quality Engineering

Testing an ML pipeline is different from testing a CRUD app: the most important outputs are probabilistic, the heaviest components take minutes and gigabytes, and “correct” is a score, not a boolean. This chapter defines what “tested” means at SyncDub, layer by layer, so that “the tests pass” is a meaningful sentence. Today it is not: `test.py`, `test_tts.py`, and `test_upload.py` are scratch scripts with no assertions, no runner, and no CI.

10.1 The four layers

Layer 1 — Unit tests (deterministic logic). Fast, no models, no network. Prime targets already in the codebase: segment grouping into context blocks (a pure-logic function with real linguistic consequences), pitch-threshold gender classification given synthetic pitch arrays, progress-state transitions, path/naming logic (the Stage-1 job-ID work), translation retry/backoff behavior with a mocked translator. Run on every PR; measured in seconds.

Layer 2 — Contract tests (stage adapters). Each stage interface (`Transcriber`, `Translator`, `Synthesizer`, `LipSyncer` — Chapter 7 §7.1) gets a contract suite that any implementation must pass: given a 5-second fixture, returns segments with monotonic timestamps; given Devanagari input, output is valid UTF-8 Marathi text; given text and a reference voice, produces a playable WAV of plausible duration; errors raise typed exceptions with stage context (Chapter 9 §9.2). Contract tests are what make model swapping (the sovereignty principle) safe *mechanically*, before evaluation makes it safe *qualitatively*. Run on every PR against fake/stub implementations; against real models nightly.

Layer 3 — Golden pipeline tests (end-to-end, small). Three to five short fixture videos (10–20 seconds: one male speaker, one female, one noisy audio, one code-switched Hindi-English) run through the entire real pipeline on a schedule and before any release. Assertions are structural and tolerant, not byte-exact: pipeline completes, every stage artifact exists, output video duration within tolerance of input, audio track non-silent, transcript non-empty and Devanagari. These catch the “someone broke stage wiring” class of failure that unit tests can’t see and eval suites are too slow for. Byte-exact comparison of media outputs is a known false-negative factory — never assert on encoded bytes.

Layer 4 — Evaluation suites (quality as a number). WER on our Hindi test set, translation adequacy, isochrony error, LSE-C/LSE-D for lip sync, and eventually automatic dub-quality score — Chapter 16 owns the definitions. The testing chapter’s rule about them: **eval scores are release gates, not dashboards.** A model or prompt change that drops the suite score below its floor does not ship, exactly as a failing unit test does not merge. This is the mechanical link between the testing culture and the quality moat.

10.2 What we deliberately do not test

Honesty about the budget: no UI pixel tests (the frontend gets Layer-1 logic tests and one smoke test that the app renders and can poll a mocked job); no load testing before there is

load (Chapter 8's ceiling analysis stands in); no fuzzing beyond input-validation basics until the attack surface is public (Chapter 12 revisits); no coverage-percentage targets, ever — coverage is a flashlight, not a KPI, and chasing it produces assertion-free tests like the ones we're replacing.

10.3 Fixtures and test data

- A `fixtures/` directory of small, *rights-cleared* clips — we cannot test a consent-first product on videos we don't have rights to (Chapter 5 eats its own cooking). Record them ourselves; a founder reading two sentences of Hindi in four acoustic conditions is an afternoon's work and a permanent asset.
- Fixtures are versioned and immutable; changing a fixture is a schema-level review event (it silently redefines every downstream score).
- Real customer content never becomes test data without the explicit consent path from Chapter 20; "it was handy" is a trust violation, not a shortcut.

10.4 CI reality

Minimal viable CI (rides Chapter 8 Stage 0): every PR runs format + lint + Layer 1 + Layer 2 (stubbed) in minutes, CPU-only. Nightly runs Layer 2 (real models) + Layer 3 on a GPU runner when one exists, CPU meanwhile — slow nightly beats absent. Release runs everything plus the Layer 4 floors. A red nightly is triaged the next morning, not silenced: a flaky golden test is a real bug in either the pipeline or the test, and both are worth a morning.

10.5 The cultural rule

A bug found by a customer that a Layer 1–3 test could have caught costs a test, not a blame. Every incident postmortem (Chapter 13) asks "which layer should have caught this?" and the fix PR includes that test. This is how the suite grows to fit *our* failure modes instead of a textbook's.

Key takeaways

- Four layers: unit (logic, every PR), contract (stage interfaces, what makes model swaps mechanically safe), golden pipeline (small end-to-end fixtures, structural assertions), evaluation (quality scores as release gates).
- Media outputs are asserted structurally, never byte-exactly; eval floors block releases exactly like failing tests.
- Fixtures are small, rights-cleared, versioned, and immutable; customer content is never casually test data.
- No coverage targets, no load tests before load, no pixel tests — the budget goes where our failures actually are.

- Every escaped bug buys the test that would have caught it.

Questions founders should ask

1. Can a new engineer run Layers 1–2 locally in under five minutes? If not, testing is still folklore.
2. When did the golden pipeline last go red, and was it triaged or silenced?
3. Which release gate has been waived recently, by whom, and is that written down?
4. Do our fixtures cover the failure modes customers actually hit (noise, code-switching, multiple speakers), or the ones that were easy to record?

Future research topics

- Record the v1 fixture set (four clips, rights-cleared) — an afternoon that unblocks Layers 2–3.
- Investigate deterministic-mode options for Whisper/TTS inference (seeds, temperature) to tighten golden-test tolerances.
- Evaluate lightweight GPU CI options (a single spot instance on a nightly cron beats a hosted-runner bill) alongside Chapter 11’s cost work.

Chapter 11: Infrastructure, GPU & Cost Standards

The founding brief asked for “GPU Standards.” The correct standard is one level up: **cost per dubbed minute (CPDM) is the governing metric of all infrastructure**, and every GPU, instance, and storage decision is a move in the CPDM game. This ordering matters because our strategy (Chapter 2 §2.3) depends on a price corridor — human dubbing far above us, our costs far below our price — and infrastructure is where that corridor is either defended or squandered.

11.1 CPDM: definition and discipline

CPDM = total marginal cost to produce one minute of reviewed, delivered, dubbed video. It decomposes into: compute (per stage), storage and egress, third-party API fees (today ₹0 because we’re using unlicensed free endpoints — a fake zero that Chapter 15’s swaps will make real and honest), and the human review minutes it triggers (Chapter 21 — yes, editor time is in CPDM; excluding it is how “AI-first” companies discover they run an agency).

Disciplines:

- **Measured, not modeled.** Stage 5 of the migration (Chapter 8) emits per-stage duration and resource use per job; CPDM is computed from real jobs weekly. Today nobody knows this number (Chapter 1 §1.5) — the first measurement is this chapter’s most urgent action.
- **Decomposed by stage.** The hypothesis is that Wav2Lip inference dominates GPU time and Whisper dominates memory, but hypotheses are not measurements. The stage breakdown decides where optimization effort goes; optimizing an already-cheap stage is theater.
- **Tracked against price.** Chapter 24 sets price per dubbed minute; the CPDM:price ratio is reviewed monthly. Margin erosion arrives quietly, one convenient instance-type upgrade at a time.

11.2 GPU standards

- **Rent, never buy, until utilization proves otherwise.** Owned hardware is justified only when measured sustained utilization would repay it inside ~18 months — a spreadsheet check in a decision record, not an instinct. Early-stage GPU ownership is how startups convert runway into depreciating paperweights.
- **Right-size per stage, not per pipeline.** ASR, TTS, and lip sync have different GPU appetites; running the whole pipeline on the GPU the *hungriest* stage needs means paying peak price for every stage. The job system’s per-stage structure (Chapter 7 §7.2) lets stages run on different instance classes once volume justifies the split — this, not microservice fashion, is the measured reason GPU stages split first.
- **Batch is our friend.** Nothing in the wedge product is latency-critical (a lecture dub returning in an hour is fine — Chapter 3 rejected real-time). That unlocks the cheap

end of every market: spot/preemptible instances with checkpoint-resume (per-stage artifacts make resumption natural), off-peak scheduling, and aggressive batching of Wav2Lip inference. Our latency tolerance is a *cost weapon*; guard it against product creep that would casually promise "results in minutes."

- **Utilization is the KPI, not the fleet.** One dashboard number: GPU-hours paid vs. GPU-hours doing work. Below ~60%, fix scheduling before adding capacity.

11.3 Model-serving standards

- Weights load once per worker lifetime, not per job — today's per-run loading of Whisper (1.5 GB) is rational for a demo script and ruinous for a service; the worker model (Chapter 8 Stage 2) amortizes it.
- The lazy-load/explicit-unload discipline from `inference_marathi.py` survives as the *within-worker* memory policy for stages that can't co-reside.
- Quantization/compression (e.g., faster-whisper, int8) are evaluated like any model change: through the Chapter 16 suite with CPDM attached. A 40% cost cut for one eval-point drop is usually a great trade — but it goes through the gate, not around it.

11.4 Storage and bandwidth standards

Video is heavy, and Indian egress fees are real money at scale:

- Artifacts stored per job with a **lifecycle policy from day one**: source and output retained per customer contract; intermediate artifacts (extracted WAVs, frame caches) deleted on job completion + a debugging grace window. Storing everything forever is a silent CPDM tax and a Chapter 12 liability at once.
- Egress discipline: signed, expiring URLs; no free unlimited re-downloads in the product's default tier; output bitrate/resolution defaults chosen for the actual viewing context (a student's phone) rather than videophile pride — with the Chapter 3 caveat that quality-crushing compression fails the "student is the user" test. Measured, again: pick the lowest bitrate that doesn't move review scores.

11.5 Environment standards

Three environments, no more: **local** (docker compose, CPU paths, stub adapters — the Chapter 8 Stage 0 deliverable), **staging** (real models, small GPU, golden tests and nightly evals live here), **production**. Infrastructure is code (even if it's a well-commented Terraform file and a bootstrap script); the console is for looking, not changing. Anything clicked into existence is one outage away from being unreproducible.

Key takeaways

- Cost per dubbed minute — including honest license fees and human review time — governs all infrastructure; measure it weekly from real jobs, decomposed by stage.

- Rent GPUs, right-size per stage, exploit our batch latency tolerance as a cost weapon (spot instances, off-peak, batching); watch utilization, not fleet size.
- Load weights per worker, not per job; quantization goes through the eval gate with CPDM attached.
- Storage has a lifecycle from day one; egress is designed, not discovered.
- Three environments, all reproducible from code.

Questions founders should ask

1. What is CPDM this week, which stage dominates it, and how has it moved month-over-month?
2. What fraction of CPDM is currently a “fake zero” (unlicensed free endpoints) and what does it become after the sovereignty swaps?
3. Has any product promise crept toward latency guarantees that would forfeit the batch-cost weapon?
4. If our biggest customer 10×’d volume next month, which line item breaks first — GPU capacity, storage, or review hours?

Future research topics

- First real CPDM measurement: instrument the current pipeline (even pre-migration) with per-stage timing and run ten representative jobs on a rented GPU vs. CPU.
- Benchmark faster-whisper/int8 Whisper against the current medium model on the Chapter 16 golden set with cost attached.
- Price out Indian-region GPU availability (major clouds vs. Indian providers like E2E/AceCloud) — data-residency pressure from Chapter 12 may constrain this choice, so do it once, jointly.

Chapter 12: Security & Compliance Standards

Customer video is sensitive by nature — a professor's face, a company's unreleased course, a government program's content — and our product additionally handles *voice prints*, which are biometric-adjacent personal data. Security here is not a compliance checkbox; it is the engineering half of the trust moat (Chapters 4–5). This chapter is sequenced like everything else in Part II: the embarrassing basics now, the enterprise posture when enterprise pulls.

12.1 Where we start (the honest list)

Today's surface, from Chapter 1 §1.2: no authentication, CORS `allow_origins=["*"]`, uploaded filename used directly as a disk path (path traversal), outputs served statically to anyone, secrets and config as scattered environment writes, no HTTPS story, no dependency auditing. None of this is surprising for a demo; all of it ends with the Chapter 8 migration, whose stages carry the fixes: job UUIDs kill filename-as-path (Stage 1), API keys and locked CORS arrive at Stage 3, signed expiring URLs replace static serving at Stage 4.

12.2 Baseline standards (apply from the first customer)

- **Tenancy isolation is a query discipline.** Every data access is scoped by `account_id` at the query layer — not filtered in application code after fetching. One shared helper builds scoped queries; direct table access outside it is a review-blocking offense. This single rule prevents the classic multi-tenant leak class.
- **Secrets management:** no secrets in git, in images, or in code (CI check); one secrets store (the platform's native one is fine); rotation is possible without a deploy.
- **Transport and at-rest:** HTTPS only, HSTS on; object storage buckets private-by-default with signed URLs (Stage 4); database encrypted at rest (a checkbox on any managed Postgres — check it).
- **Input handling:** uploads validated by content type and size cap before processing; media processed by ffmpeg/MoviePy is a real parsing attack surface, so workers run as unprivileged users in containers with no outbound network access except what the stage needs — the `clone_bridge.py` isolation habit, generalized into a security boundary.
- **Dependency hygiene:** pinned versions (Chapter 9 §9.5) plus an automated vulnerability scan (pip-audit / npm audit in CI). ML dependency trees are enormous; scanning is not optional at our dependency count.
- **Least privilege for humans too:** production access is named, logged, and minimal. At two engineers this feels silly; it is precisely the habit that cannot be retrofitted at twenty.

12.3 Data protection (DPDP/GDPR posture)

India's DPDP Act is our home regime; GDPR is the template enterprise buyers ask about. Our posture, engineered rather than promised:

- **Data inventory by design:** the job schema (Chapter 7 §7.5) *is* the inventory — every artifact traces to an account, a purpose (the job), and a timestamp. What most companies reconstruct in panic before an audit, we get from the architecture.
- **Deletion that actually deletes:** "delete my data" resolves to a deletable per-job storage prefix (Stage 4) plus row deletion, including intermediate artifacts and — critically — **voice reference samples** (`reference_voice.wav` today), which are the most sensitive artifact we hold. Deletion is tested in staging like any feature. The audit ledger's *hashes* (Chapter 5 §5.4) survive deletion by design: proving what we made requires no retained media.
- **Retention by contract, not by default-forever:** every artifact class has a stated retention period; the lifecycle policy (Chapter 11 §11.4) enforces it, so compliance and cost control are one mechanism.
- **Consent records are data too:** attestations (Chapter 5 §5.2) are retained as long as the legal exposure they answer, independent of media deletion.
- **Data residency:** wedge customers (Indian education/government) will increasingly require in-India processing and storage. Choose regions accordingly *now* (jointly with the Chapter 11 GPU sourcing decision) — residency is nearly free as a starting constraint and brutal as a migration.

12.4 Security decision-making

How security decisions are made (per the founding brief's demand):

- Security sits inside the **trust layer** of the decision stack (Chapter 3) — above quality, cost, and speed. Concretely: a launch blocked on a tenancy-isolation bug stays blocked, whatever the demo calendar says.
- **Threat-model the deltas.** Big-bang security reviews rot; instead, any PR touching auth, tenancy scoping, upload handling, URL signing, or the consent/audit schema carries a "what could this leak or forge?" paragraph in the description, and gets the trust-priority review of Chapter 9 §9.3.
- **Assume breach in design:** signed URLs expire; keys rotate; workers are sandboxed; blast radius per stolen credential is enumerable. We are a small target today; automated scanning doesn't care about company size.
- **Disclosure readiness:** a `security.txt` and a monitored contact address cost an hour and signal seriousness; when a researcher finds the inevitable bug, we want the report, not the tweet.

12.5 What we defer, explicitly

SOC 2 / ISO 27001 certification waits for a named deal that requires it (Chapter 23 sequences this) — but every §12.2–12.3 habit is chosen to *be* the evidence those audits want, so certification becomes paperwork over practice, not a rebuild. Pen tests wait for a public API surface. A bug-bounty program waits for the security team that can triage it.

Key takeaways

- Security is the engineering half of the trust moat; its fixes ride the Chapter 8 migration stages rather than forming a separate project.
- Tenancy isolation by query discipline, sandboxed media workers, pinned-and-scanned dependencies, secrets out of git — the baseline from customer one.
- DPDP/GDPR posture is architectural: the job schema is the data inventory, deletion is a tested feature (voice samples especially), retention equals lifecycle policy, residency is chosen now.
- Security decisions live in the trust layer of the decision stack; threat-modeling happens on deltas, in PRs, not in annual ceremonies.
- Certifications are deferred but pre-paid: today's habits are tomorrow's audit evidence.

Questions founders should ask

1. Can any query in the codebase reach another account's rows today? Who checked?
2. Does deletion actually remove voice reference samples and intermediates, and when was that last tested?
3. Which stage of the Chapter 8 migration is the current security posture waiting on, and is anything customer-facing shipping ahead of it?
4. If a security researcher emailed us tonight, does the address exist and does anyone read it?

Future research topics

- DPDP Act implementation rules: track the actual notification schedule and consent-manager requirements; assign the same owner as Chapter 5's regulatory watch.
- Evaluate container sandboxing depth for media workers (gVisor/firecracker vs. plain containers) once there is a public upload surface.
- Draft the data-processing agreement (DPA) template early — enterprise deals (Chapter 23) will ask, and legal drafting has a long lead time.

Chapter 13: Reliability, Releases & Failure Analysis

SRE-lite: the smallest set of reliability practices that a team of two can actually sustain, chosen so that each one scales rather than needing replacement. The organizing insight for *our* product: *a batch pipeline fails differently than a website.* Users don't experience downtime; they experience **jobs that never finish, finish wrong, or finish silently**. Reliability engineering at SyncDub is therefore job-completion engineering first and uptime engineering second.

13.1 What reliability means here

Ranked reliability objectives (our SLO seeds, formalized when customers hold contracts — Chapter 23):

1. **No lost jobs.** Every accepted job terminally reaches `done` or `failed(stage, cause)` — never limbo. Today's `Popen` fire-and-forget violates this by design; the queue with heartbeats (Chapter 8 Stage 2) is the fix, and "worker died mid-job" is the first failure mode it must handle (retry or resume from the last stage artifact).
2. **No silent wrongness.** A job that completes with an empty transcript, silent audio, or a 3-second output for a 10-minute input is worse than a failure — it reaches a customer. The golden-test structural checks (Chapter 10 Layer 3) run as **post-conditions on every production job**, not just in CI; violations flip the job to `failed` rather than `done`. This is cheap and catches the class of bug that erodes trust fastest.
3. **Honest progress.** The progress a customer sees reflects job state truthfully, including "queued behind N jobs" and "failed at translation, will retry." The current UI's optimistic percentage is demo culture (Chapter 1); a truthful queue position retains customers better than a stuck 60%.
4. **API availability.** Last, deliberately: the API being briefly down delays *submissions*; the pipeline being wrong damages *outputs*. Effort follows harm.

13.2 Release standards

- **Releases are boring, small, and reversible.** Deploy from `main` via CI only (no laptop deploys — reproducibility is Chapter 11 §11.5's environment rule applied to people); every release must be revertible by redeploying the previous image, which forbids coupled irreversible migrations — schema changes ship additively first (expand), code moves over, cleanup follows (contract).
- **Model releases are releases.** Swapping a model, changing a prompt/parameter (even `beam_size`), or updating weights goes through the same gate as code: eval floors (Chapter 10 Layer 4) plus golden tests, with the model version recorded on every job it processes (the registry — Chapter 15 — makes this queryable). "It's just a model file" is how quality regressions ship unlogged.
- **Stage or shadow when the change is scary.** Before a swapped stage serves customers, run it in shadow — same inputs, outputs stored but not delivered, scores

compared (Chapter 19 formalizes this). Our per-stage artifact design makes shadow runs nearly free to implement; use them instead of courage.

- **Release cadence is a habit, not an event.** Small releases, at least weekly, even pre-customers. Teams that release rarely get bad at releasing exactly when it starts mattering.

13.3 Monitoring standards

The minimum dashboard that answers “is the system healthy?” at a glance:

- **Jobs:** submitted / completed / failed per day, failure rate *by stage*, p50/p95 job duration, queue depth and oldest-job age (the early-warning metric — a growing oldest-age means workers are dying or drowning).
- **Quality canaries:** rolling rate of post-condition violations (§13.1.2) and, once Chapter 16 lands, rolling automatic quality score — a slow quality drift is an incident nobody pages for, which is why it gets a chart.
- **Cost:** CPDM weekly (Chapter 11) — cost regressions are reliability regressions of the business.
- **Alerts follow the pager rule:** alert only on conditions someone will act on *now* (no worker heartbeat, failure-rate spike, oldest-job age breach, disk/queue saturation). Everything else is a chart reviewed weekly. At two engineers, a noisy pager is self-DDoS; alert fatigue kills more reliability than missing metrics do.

13.4 Incidents and failure analysis

The founding brief asked “how failures are analyzed.” The loop:

1. **During:** one person owns the incident (at current size, whoever noticed — announce it); mitigate first (usually: pause the queue, revert the release, re-run affected jobs), diagnose second. Customer-affecting incidents get a proactive message before customers notice — in the wedge market, “we caught a quality issue and are re-running your jobs free” *builds* trust rather than spending it.
2. **After (within 48h):** a blameless postmortem, one page, in `docs/postmortems/`: timeline, root cause, customer impact, and three mandatory questions — *Which test layer should have caught this?* (buys a test, Chapter 10 §10.5), *Which alert should have fired?* (buys an alert or explicitly declines one), *Which handbook chapter was wrong or silent?* (buys an edit — this is how the handbook stays true, per the index’s living-document rule).
3. **Blameless means causes, not exemption from ownership.** The postmortem names systems and gaps, never villains; the follow-up items get owners and land in the 20% debt/interrupt capacity (Chapter 3 §3.5) rather than a someday-list.

13.5 On-call, sized honestly

Pre-customers: no pager; the morning triage of nightly runs (Chapter 10 §10.4) is the whole practice. First customers: business-hours responsiveness, founders carry it. Contracted SLAs: that is a *sales decision that buys infrastructure* — an SLA promise prices in the on-call rotation, the staging environment parity, and the redundancy it requires (Chapter 23 makes this pricing explicit). Never sign reliability the roster can't deliver.

Key takeaways

- Batch reliability = job-completion reliability: no lost jobs, no silent wrongness (production post-conditions), honest progress, then API uptime — in that order.
- Releases: small, weekly, reversible, expand-then-contract migrations; model changes are releases with eval gates and recorded versions; shadow runs replace courage.
- Monitor jobs, quality canaries, and CPDM; page only on actionable conditions — alert fatigue is the real enemy at this size.
- Every incident buys a test, an alert (or its documented refusal), and a handbook edit; blameless names causes, not villains.
- SLAs are priced promises — sign nothing the current roster can't answer at 2 a.m.

Questions founders should ask

1. Can a job be lost in limbo today, and would we know? (Until Chapter 8 Stage 2: yes, and no.)
2. What percentage of last month's completed jobs would have failed the production post-conditions, had they existed?
3. When did we last practice a revert — not discuss one, practice one?
4. Which postmortem follow-up items are past due, and did the 20% capacity actually absorb them?

Future research topics

- Define the v1 post-condition set from the golden-test assertions (duration tolerance, non-silence, transcript sanity) and wire them into the current pipeline even pre-migration — cheap and immediately valuable.
- Pick the minimal observability stack (managed Grafana/Prometheus vs. a hosted product) with a CPDM-conscious eye; a one-page decision record.
- Draft the customer-facing incident-communication template now, in Marathi and English — writing it calmly precedes needing it urgently.

Chapter 14: Technical Debt & Refactoring Protocol

Our house style ships ugly bridges on purpose (Chapter 3), which means we generate debt on purpose — so we need a sharper repayment discipline than teams that pretend not to borrow. The core idea of this chapter: **debt is repaid on triggers, not on calendars.** "Refactoring sprints" scheduled by guilt repay the wrong debts at the wrong time; trigger-based repayment repays exactly the debt that is about to hurt.

14.1 What counts as debt (and what doesn't)

Debt is a *known* gap between how the system is and how it needs to be, **with an owner-visible record**. The `clone_bridge.py` subprocess hop is debt done right: contained, documented, working. The hardcoded `D:/Miniconda3` ffmpeg path is debt done wrong: undocumented, wide (it breaks every non-founder machine), discovered rather than declared.

Not debt: code that is merely old, style you'd write differently today, or a library that has a trendier alternative. Rewriting working, contained, measured code is not repayment — it is *taking out a new loan to redecorate*, and the measured-evolution principle (Chapter 7 §7.4) blocks it.

14.2 The debt register

One file: `docs/debt.md`. Each entry, three lines: what the shortcut is, what it will break or block, and its **trigger** — the observable condition that makes repayment mandatory. Entries are added in the same PR that ships the shortcut ("ship the ugly bridge *with a debt entry*" — Chapter 3). No tickets rotting in a tracker; the register is short precisely because it is curated: if an entry has no plausible trigger, it isn't debt, delete it.

Seed entries, from the current codebase:

Debt	Trigger that forces repayment
Direct model imports, no stage interfaces (`inference_marathi.py`)	First model swap attempt — which licensing has already scheduled (Ch. 15)
Global `progress.json`, no job identity	Second concurrent user — i.e., first real customer trial
`Popen` fire-and-forget, no queue	First lost-job incident, or first customer, whichever first
No auth, CORS `*`, filename-as-path	Any non-demo deployment reachable from the internet
Unpinned dependencies, two undocumented conda envs	First non-founder engineer's first day (Ch. 6 hire #1)
Frontend's 714-line `App.js` single component	First real frontend feature beyond the demo flow (the editor, Ch. 21)
No LICENSE file	First external code reader — investor diligence or open-source decision (Ch. 26)

Note what the table reveals: most of our debt triggers fire on the same three events — *first customer, first hire, first model swap*. That is the honest definition of our current stage, and it is why Chapter 8's migration and Chapter 15's swaps are the roadmap rather than items on it.

14.3 Repayment rules

- **Trigger fires → repayment enters the current quarter**, funded from the 20% reserve (Chapter 3 §3.5). A fired trigger that gets re-snoozed must be re-snoozed *in writing* in the register, with the new trigger — silent snoozing is how registers become fiction.
- **Repay at the touch.** When feature work enters a debted area, the PR either repays the debt or consciously extends it (one line in the register). What it may not do is silently build a second storey on the known-bad foundation — that converts debt into architecture.
- **Repayment ships with the same gates as features:** tests for the new shape (Chapter 10), before/after measurement where performance is claimed (Chapter 7 §7.4), and a register deletion in the same PR. A refactor that can't say what it fixed isn't done.
- **Containment work counts as repayment.** Sometimes the right move isn't removing the ugliness but driving it deeper (Chapter 7 §7.3) — wrapping the bridge in an adapter, isolating the global into one module. Cheaper, and often all the trigger actually demands.

14.4 The debt conversation with product pressure

The recurring failure mode in every company: debt repayment loses the argument against features, quarter after quarter, until an incident wins the argument catastrophically. Our structural defenses: the trigger system makes repayment *non-discretionary* (it's not competing in the prioritization meeting — the trigger already decided), the 20% reserve means it has a budget without renegotiation, and the register's "what it will break" line converts engineer unease into founder-legible risk. The founder's half of the bargain: never spend the 20% reserve on features in a good month. Reserves spent in good months don't exist in bad ones.

14.5 The rewrite question

Someday someone will propose rewriting the pipeline/backend/frontend wholesale. The standard: rewrites are approved only when (a) the debt register shows a *cluster* of fired triggers in one area that incremental repayment demonstrably can't clear, (b) the rewrite has a staged, each-stage-ships-alone plan (the Chapter 8 shape — which is itself the approved rewrite of the backend, done right), and (c) a decision record names what is *kept* (Chapter 8 §8.4's discipline). Big-bang rewrites with a cutover day are rejected categorically; they are how working companies produce non-working software.

Key takeaways

- We borrow deliberately, so we repay deliberately: every shortcut ships with a register entry and an observable trigger; calendars and guilt schedule nothing.
- The current register's triggers cluster on first customer, first hire, and first model swap — which is why Chapters 8 and 15 are the roadmap.
- Repay at the touch, fund from the protected 20% reserve, delete the entry in the repaying PR, and let containment count.
- Old-but-working code isn't debt; rewrites need fired-trigger clusters and staged plans, never cutover days.

Questions founders should ask

1. Does `docs/debt.md` exist and match reality, or is this chapter still describing an intention?
2. Which triggers have already fired without repayment starting — and was the snooze written down?
3. Was the 20% reserve spent on features in any recent month? Who noticed?
4. Is any current PR quietly building on a register entry without extending or repaying it?

Future research topics

- Create `docs/debt.md` from the §14.2 seed table — a fifteen-minute task that makes the whole protocol real.
- Add a PR-template checkbox (“touches a debted area? register updated?”) once CI exists (Chapter 9).
- Revisit the register quarterly against the incident log (Chapter 13): incidents in un-registered areas mean our debt radar has a blind spot worth studying.

PART III — AI RESEARCH OS: HOW AI SYSTEMS THINK

Chapter 15: The Model Layer

The founding brief demanded: “Never lock the company to today’s AI models. Design for the next generation of AI.” This chapter is the mechanism. The model layer has four parts — stage interfaces, a registry, a routing policy, and a replacement pipeline — and one immediate mission: replacing every legally-blocked component identified in Chapter 1 §1.3.

15.1 Stage interfaces (recap and contract)

From Chapter 7 §7.1: `Transcriber`, `Translator`, `Synthesizer`, `LipSyncer` (later `Diarizer`, Chapter 18). Each contract specifies inputs/outputs in *domain terms* (audio + language → timed segments; segments + target language → translated segments; text + voice spec → audio; video + audio → video), typed failures, and the metadata every implementation must return: model identifier, version, processing cost hints. The contract-test suites (Chapter 10 Layer 2) enforce mechanically that implementations are interchangeable.

The subtle design rule: **interfaces encode the *task*, not the current model’s shape.** Whisper returns rich segment objects; a future streaming ASR might not. The interface owns the segment schema (Chapter 7 §7.5); adapters translate whatever their model emits into it. When a next-generation model collapses two stages into one (e.g., direct speech-to-speech translation), it implements *both* interfaces behind a combined adapter — the pipeline shape survives even when models merge stages.

15.2 The model registry

Every model usable in production has a registry entry (a directory of YAML files under `models/registry/` is entirely sufficient — the registry is a discipline, not a product):

- identity: name, version, weights hash or API version
- **license status and the decision record link** (the sovereignty gate — Chapter 7 §7.1; no entry, no deploy)
- eval scores on the current golden suites, with dates (Chapter 16)
- cost profile: per-minute compute on reference hardware (feeds CPDM, Chapter 11)
- operational notes: memory footprint, environment/bridge requirements, known failure modes

Every job records which registry entries processed it (Chapter 13 §13.2’s model-releases-are-releases rule). This is what makes questions like “which customers received outputs from the buggy TTS version?” answerable in one query instead of one archaeology.

15.3 The sovereignty swaps (the immediate agenda)

Current stage-by-stage replacement agenda, each item a decision-record-to-be with candidates to benchmark, not conclusions:

Stage	Today (status)	Candidates to evaluate
ASR	Whisper medium (MIT — **legal, keep**)	faster-whisper (cost), large-v3 (quality), IndicWhisper-style Indic fine-tunes
Translation	deep-translator scraping Google (**ToS-blocked**)	IndicTrans2 (AI4Bharat, open, Indic-specialized — the obvious first benchmark), NLLB, licensed cloud MT as fallback tier
TTS/cloning	Edge-TTS (unlicensed endpoint) + XTTS v2 (**non-commercial**)	Licensed commercial APIs; permissive open models with Marathi support (a moving field — survey at benchmark time, not in this document)
Lip sync	Wav2Lip (**non-commercial, competitor-owned**)	MuseTalk and successors (verify licenses individually), commercial lip-sync APIs, licensed-Wav2Lip negotiation as a priced option

Two honest notes. First, ASR is already legal — the licensing crisis is really a translation/TTS/lip-sync crisis, which usefully narrows the work. Second, the lip-sync column is the hardest: the field's strongest models trend non-commercial, so the realistic v1 outcome may be a *commercial API* at honest CPDM cost (Chapter 11's "fake zero" made real) while open alternatives mature. The registry's cost field exists precisely so that trade can be made with numbers.

Swap order follows exposure: translation first (ToS-blocked *and* the best open replacement exists), TTS second, lip sync third (hardest, and non-commercial use remains legal while revenue is zero — but it must be swapped before the first rupee).

15.4 Routing policy

Once the registry holds more than one live implementation per stage, routing chooses per job. Routing inputs: customer tier (Chapter 24's quality/cost tiers), content properties (duration, noise, language pair), and cost state. The v1 policy is a lookup table in code — explicitly not an ML system; a routing decision must be explainable to a customer in one sentence ("your tier uses X"). Learned routing is a someday-entry gated on the eval suite being able to *judge* it.

15.5 The replacement pipeline (how models enter and exit)

The permanent process, so next-generation adoption is routine rather than heroic:

1. **Scout** — candidate spotted (paper, release, license change). Anyone can nominate; a registry stub records it.

2. **License gate** — before any engineering: license read, decision record drafted. Kills most candidates cheaply.
3. **Adapter + contract tests** — implement the stage interface; pass Layer 2. Typically days, thanks to the interface work.
4. **Benchmark** — run the Chapter 16 suites; record scores and cost in the registry. No benchmark, no opinion: advocacy without scores is noise (Chapter 3's "measured beats impressive").
5. **Shadow** — run beside production on real jobs (Chapter 13 §13.2); compare automatic scores and spot-check with the founding linguist (Chapter 6 hire #3).
6. **Promote / archive** — routing table updated by decision record; the loser stays in the registry with its scores, because today's loser is next year's re-benchmark shortcut.

Exit follows the same path in reverse: a model leaves routing before it leaves the registry, and never leaves the registry's history — job records point at it forever (audit, Chapter 5 §5.4).

Key takeaways

- Four mechanisms — task-shaped interfaces, a YAML registry with license/eval/cost per model, explainable routing, and a six-step replacement pipeline — turn "never lock to today's models" from a wish into a process.
- Interfaces encode tasks, not current model shapes; merged next-gen models implement multiple interfaces behind one adapter.
- The immediate agenda: translation swap first (IndicTrans2 benchmark), TTS second, lip sync third — all before the first rupee of revenue.
- Every job records its models; every model records its license, scores, and cost. No registry entry, no deploy; no benchmark, no opinion.

Questions founders should ask

1. Which stage would take the longest to swap today, and is that the same answer as last quarter (i.e., are the interfaces actually landing)?
2. Has the IndicTrans2 benchmark run yet? It is the single cheapest de-risking action on the legal agenda.
3. What would we do if the chosen lip-sync vendor doubled prices — does the registry hold a scored, ready alternate?
4. Are job records actually carrying model versions, or is that still aspirational?

Future research topics

- Run the translation bake-off (deep-translator vs. IndicTrans2 vs. one licensed API) on the golden set — this both fixes the worst legal exposure and forces the Chapter 16 metrics into existence.

- Survey the current Marathi-capable TTS field with licenses in a single scouting doc (step 1–2 of the pipeline, batched).
- Watch for speech-to-speech translation models maturing — they merge three of our stages, and §15.1's combined-adapter rule is the plan for that day; a small prototype ahead of need would validate it.

Chapter 16: The Evaluation OS

You cannot swap, buy, price, or improve what you cannot measure. Every hard decision in this handbook — the sovereignty swaps, quantization trades, routing tiers, release gates, even competitive positioning — resolves into “run the eval suite and compare.” This chapter builds that suite. It is also the strategic centerpiece: Chapter 4 named quality-metric ownership a moat; the company that defines how Indic dub quality is measured referees the category.

16.1 What a dub score must capture

A dubbed video can fail in six distinguishable ways, so the metric is a six-axis vector before it is a single number:

1. **Transcription fidelity** — did we hear the Hindi right? Metric: WER against reference transcripts. Fully automatic once references exist.
2. **Translation adequacy & fluency** — is the Marathi faithful and natural? Metrics: chrF/BLEU against reference translations for regression detection, plus periodic human adequacy ratings — automated MT metrics are weakest exactly where our moat is (idiom, code-switching, honorifics), so the linguist’s panel (Chapter 6 hire #3) calibrates them rather than being replaced by them.
3. **Voice quality & similarity** — does it sound human, and (cloning path) like the speaker? Metrics: MOS panels for naturalness; speaker-embedding cosine similarity for cloning fidelity. Semi-automatic.
4. **Isochrony** — does speech fit the original timing? Metric: per-segment duration error distribution (Chapter 17 defines it precisely). Fully automatic, and currently *unmeasured while probably being our worst axis*.
5. **Lip sync** — do the lips match the audio? Metrics: LSE-C/LSE-D via SyncNet — the evaluation harness for this is already sitting unused in `backend/Wav2Lip_repo/evaluation/scores_LSE/` (Chapter 1 §1.5). Fully automatic.
6. **Delivery integrity** — structural sanity (duration, non-silence, A/V mux). Already specified as production post-conditions (Chapter 13 §13.1).

The **composite dub score** is a weighted roll-up for dashboards and marketing; *decisions* use the vector, because a composite hides exactly the trade-offs that matter (a TTS swap that gains naturalness but breaks isochrony must be visible as that trade).

16.2 Golden sets

Three tiers of curated test data, each answering a different question:

- **The smoke set** (~5 clips, the Chapter 10 fixtures): does the pipeline work? Runs everywhere, cheap.

- **The benchmark set** (30–50 clips, the real asset): stratified across our actual failure surface — male/female speakers, clean/noisy audio, slow lecture/fast conversation, pure Hindi/heavy code-switching, with reference transcripts and reference translations produced by the founding linguist. This is what model swaps and releases are judged on. Building it is *the* prerequisite for the entire Chapter 15 agenda and is measured in linguist-days, not engineer-months.
- **The frontier set** (grows from the flywheel): every correction cluster from real customer content (Chapter 20) becomes a test case. This tier is the moat mechanically realized — our benchmark converges on the failures that occur *in production Indic content*, which no competitor can replicate without our traffic.

Governance: golden sets are versioned and immutable per version (changing the test redefines every historical score — Chapter 10 §10.3's rule); reference answers get the same senior review as schema changes; scores always cite the set version.

16.3 Automatic quality scoring in production

The same axes, run on *every production job* (not just releases), at the automatic-only level: WER proxies (ASR confidence), duration-error stats, LSE scores, embedding similarity. Three consumers:

1. **Review triage** (the operational payoff): jobs above a confidence threshold ship with light review; below it, full editor pass. This is what makes human review scale sub-linearly with volume — the economics of Chapter 21 depend on this mechanism.
2. **Drift detection**: rolling score charts on the reliability dashboard (Chapter 13 §13.3) catch slow degradation no single job reveals.
3. **The flywheel's sensor**: score-vs-correction data (did editors fix what the scorer flagged?) continuously validates the scorer itself — the meta-loop that makes the metric trustworthy enough to sell against.

16.4 Benchmarking discipline

Rules that keep scores honest, several already broken once each in our own history:

- **Same set, same conditions, or it isn't a comparison.** The `beam_size=2` choice in `inference_marathi.py` was a real measured trade-off (good) recorded only as a code comment (bad). Benchmark results live in the registry (Chapter 15 §15.2) with dates and set versions.
- **No demo-driven evaluation.** A model that "looked great on the video we tried" has a score of undefined. The competition-demo instinct (Chapter 1) is our native bias; the suite is its antidote.
- **Cost rides along.** Every benchmark run records CPDM impact next to quality (Chapter 11 §11.3) — a quality-only leaderboard invites unaffordable winners.
- **Humans are calibrated, not assumed.** MOS/adequacy panels use fixed rubrics, multiple raters, and inter-rater checks; a panel that can't agree with itself can't judge

a model.

16.5 The public benchmark (the moat move)

Once the benchmark set is mature and our scores are respectable: publish it — the set (rights-cleared tier), the metrics, the methodology, and a leaderboard scoring every tool in the market including us. Referees are hard to displace; challengers must either compete on our axes (where Indic depth favors us) or argue with the methodology (attention we welcome). Timing gate, stated coldly: publish only when we win or place credibly on our own benchmark — publishing a leaderboard we lose is competitor marketing at our expense. Chapter 25 owns the launch; this chapter owns the integrity that makes it defensible.

Key takeaways

- Dub quality is a six-axis vector (ASR, translation, voice, isochrony, lip sync, integrity); composites are for dashboards, vectors are for decisions.
- Three golden tiers: smoke (works?), benchmark (30–50 stratified clips with linguist references — the prerequisite for all model swaps), frontier (correction-derived, the moat made mechanical).
- Automatic scoring runs on every production job, triaging review effort, catching drift, and validating itself against editor corrections.
- Benchmarks are versioned, cost-annotated, and demo-proof; human panels are calibrated instruments.
- The endgame is publishing the category's benchmark — after we can win it.

Questions founders should ask

1. Does the benchmark set exist yet with linguist-made references? Every week it doesn't, every model decision is being made on vibes.
2. What are our own LSE-C/LSE-D numbers? (The tool ships in our repo; ignorance here is a choice.)
3. Is review triage actually driven by scores yet, and what is the false-confidence rate (shipped-with-light-review jobs that customers then flagged)?
4. Which axis is weakest this quarter, and does the roadmap reflect that or the loudest anecdote?

Future research topics

- Stand up the LSE scoring harness from `Wav2Lip_repo/evaluation/` on current outputs — the fastest path to any real number on any axis.
- Evaluate reference-free MT quality estimators (COMET-QE class) for the production scorer, validated against the linguist's ratings.

- Design the composite-score weighting *with customers*: what educators rate as “bad dub” should set the weights, not internal intuition.

Chapter 17: The Isochrony Problem

Isochrony — making translated speech occupy the same time as the original — is the difference between a dub and a voiceover, and it gets its own chapter because it is SyncDub's central research problem: unsolved in our pipeline, under-solved in most competitors', and decisive for perceived quality. A viewer forgives a slightly odd word choice; they do not forgive a speaker whose mouth stops while the voice continues, or Marathi audio drifting seconds behind the visual gestures it belongs to.

17.1 Why it's hard, and what our pipeline does today

Languages differ in information density: a Hindi sentence and its Marathi translation take different times to say, and the difference varies per sentence. Today's pipeline (`inference_marathi.py`) transcribes with timestamps, translates in context blocks, synthesizes each block at the TTS engine's natural pace, concatenates, and hands the result to Wav2Lip. Nothing constrains synthesized duration to source duration. Wav2Lip then hides part of the sin — it re-animates lips to *whatever* audio it receives — which is exactly why the problem is invisible in demos and corrosive at scale: the lips match the words, but the words no longer match the scene. Gestures, slide changes, scene cuts, and pauses drift out of correspondence, and total video length forces either audio truncation or trailing silence.

The honest statement: **we currently rely on our lip-sync stage to disguise a timing problem we haven't measured.** (Chapter 16 §16.1 axis 4 — probably our worst axis, definitely our least known.)

17.2 The metric first

Per the house rule (measured beats impressive), the metric precedes the solutions:

- **Per-segment duration error**: `(dubbed_duration − source_duration) / source_duration`, as a distribution, not an average — one +80% segment ruins a lecture whose mean error is 3%.
- **Cumulative drift**: absolute offset between source and dub timelines over the video — the metric that catches "each segment is fine, the sum is not," which truncation-or-silence failures grow from.
- **Rate deviation**: how far any speed adjustment pushed speech from natural tempo (a +25%-sped-up segment scores perfectly on duration and sounds like an auctioneer; this metric keeps the fix honest).

All three are computable from artifacts the pipeline already produces (Whisper timestamps, synthesized audio lengths). This is days of work and should precede *any* solution investment.

17.3 The solution ladder

Ordered by cost; each rung ships value alone, and the metric decides how far up we climb:

1. **Segment-anchored placement (fix the drift).** Stop concatenating; place each synthesized block at its source timestamp, absorbing small overruns into pause gaps between segments. Eliminates cumulative drift entirely without touching synthesis quality — the ladder's best value-for-effort rung and the immediate next step.
2. **Rate adjustment within tolerance.** Time-stretch synthesized audio toward target duration, capped (±10–12% is generally imperceptible; the rate-deviation metric enforces the cap). Edge-TTS accepts a rate parameter natively; XTTS output can be stretched post-hoc with standard DSP. Cheap, safe inside the cap, ugly beyond it.
3. **Length-aware translation.** The interesting rung, and where Indic depth (Chapter 4) becomes real: generate translations under a syllable/duration budget. Mechanisms: prompt-or-constraint-based length control in the MT stage, or an LLM rephrasing pass ("same meaning, ~20% shorter Marathi") applied selectively to segments the metric flags. This is a *translation* fix for a *timing* problem — the insight most pipelines miss, and one our editor can also crowdsource ("shorten this segment" as a one-click correction feeding the flywheel).
4. **Duration-conditioned synthesis.** TTS that natively targets a duration (the research frontier). We adopt it when the model layer scouts it (Chapter 15 §15.5), not build it.
5. **Pause & prosody engineering.** Redistribute source-speech pauses (Hindi lecturers pause a lot — usable budget) and match emphasis timing. Mostly heuristics on data we already have; polish, not foundation.

The likely stable configuration for the wedge product: rungs 1+2 always on, rung 3 triggered by the metric on offending segments, rung 4 awaited, rung 5 opportunistic.

17.4 Interactions the metric must arbitrate

Isochrony trades against the other five axes explicitly: harder rate adjustment (worse voice naturalness), shorter translations (risk to adequacy), tighter timing (better lip-sync scores, since Wav2Lip stops covering gaps). This is the canonical example of why Chapter 16 insists on the score *vector* — an isochrony fix that silently costs adequacy must surface as that trade in the release gate, and the weighting question ("what does the *student* notice more?") is answerable by the customer-calibrated composite, not by engineers arguing.

17.5 Strategic note

Isochrony leadership is unusually moat-compatible: it is measurable (we can *prove* superiority on the public benchmark, Chapter 16 §16.5), it compounds through the flywheel (every editor re-timing is training signal for rung 3's selective rephrasing), and it is Indic-specific in its details (syllable-timing patterns, honorific length inflation) exactly where global players under-invest. If SyncDub becomes known for one measured thing, "dubs that keep time" is the right thing.

Key takeaways

- Isochrony is the gap between dub and voiceover; today we don’t control it, don’t measure it, and let Wav2Lip disguise it.
- Three metrics (segment duration-error distribution, cumulative drift, rate deviation) are computable from existing artifacts — build them before any fix.
- The five-rung ladder: anchor placement, capped rate adjustment, length-aware translation (the Indic-depth rung), duration-conditioned synthesis (adopt, don’t build), pause engineering. Rungs 1–2 are the immediate agenda.
- Every isochrony gain is a potential adequacy/naturalness trade; the eval vector arbitrates, weighted by what students actually notice.
- “Dubs that keep time” is the single best candidate for our provable, benchmarkable, flywheel-compounding technical identity.

Questions founders should ask

1. What is our duration-error distribution on the benchmark set? (Until answered, this chapter is theory.)
2. Has rung 1 shipped? It requires no research, no new models, and fixes the worst failure class.
3. Are editors re-timing segments in practice, and is that signal being captured in the correction schema (Chapter 4’s research topic) for rung 3?
4. Do competitor outputs drift? (Run their trials through *our* drift metric — cheap competitive intelligence and benchmark validation at once, per Chapter 2’s research topics.)

Future research topics

- Implement the three metrics and baseline the current pipeline — the prerequisite for everything above.
- Prototype rung 3 with an LLM rephrase pass on flagged segments; measure adequacy cost with the linguist panel.
- Study syllable-rate statistics for Hindi vs. Marathi on our corpus to set principled rate-adjustment caps, replacing the generic ±10–12% with Indic-specific numbers.

Chapter 18: Multi-Speaker & Emotion

Today's pipeline assumes one speaker per video: Librosa computes a single median pitch, picks one gender, one voice, and Wav2Lip re-animates whichever face it detects. That assumption fits the wedge (single-lecturer education content) and breaks on everything after it — interviews, panel lectures, dramas, corporate videos with multiple presenters. This chapter is the roadmap from one voice to many, and from words to feeling. It is deliberately *sequenced behind* the wedge: nothing here outranks Chapters 15–17 until customers bring multi-speaker content — but the schema decisions that make it cheap later are made now.

18.1 Schema now, features later

The expensive mistake would be baking "one speaker per job" into the data model the way the demo baked it into the code. The rule: **the segment schema (Chapter 7 §7.5) carries a `speaker_id` from v1**, defaulting to a single speaker. Cost today: one column. Alternative cost later: migrating every table, artifact, correction record, and eval score to a concept they were built without. Same logic as tenancy (Chapter 8): identity columns are cheap early and archaeology late.

18.2 The multi-speaker ladder

1. **Diarization** — who spoke when. A `Diarizer` stage interface (Chapter 15 §15.1's planned fifth interface) assigning `speaker_id` to segments; mature open models exist (pyannote and successors — license-gate them like everything else, Chapter 15 §15.5 step 2). Diarization errors are *catastrophic* downstream (a voice-swap mid-sentence is worse than any single-voice compromise), so the eval suite gets a diarization axis (DER — diarization error rate) before the feature ships, and low-confidence diarization routes to the editor rather than to production.
2. **Per-speaker voice assignment.** The current pitch-median gender detection generalizes: per-cluster profiling, per-speaker stock voice or clone (each clone requiring its own consent attestation — Chapter 5 §5.2 already speaks per-speaker precisely for this future). The editor shows the speaker map for confirmation; a human glance at "Speaker A / Speaker B, these voices" is cheap insurance against the catastrophic error class.
3. **Per-face lip sync.** The genuinely hard rung: matching *which face on screen* is speaking (active speaker detection) and re-animating only that face. Wav2Lip re-animates naively; multi-face content needs face tracking + speaker-face association. This rung stays research-flagged until a real customer segment demands it — dubbing a panel with correct per-face sync is film-industry territory, and film is not the wedge.

18.3 Emotion preservation

The founding brief listed “Emotion Preservation” as a bullet. The honest breakdown of what it actually is, easiest first:

1. **Prosodic carryover in cloning.** XTTS-class models already transfer some speaker affect from the reference sample; today we feed one 10-second reference (`extract_reference_sample`) chosen to avoid intro music, not to represent emotional range. Cheap upgrade: reference selection guided by the content (calm reference for calm lectures) — heuristic, measurable via MOS.
2. **Segment-level emotion tags.** Classify source segments (neutral/emphatic/questioning/excited — small taxonomy, education-content-appropriate) and pass tags to TTS engines that accept style controls. The tag also lands in the segment schema (one more column now, per §18.1) so editors can correct it — flywheel signal for whatever model eventually consumes it.
3. **Emphasis alignment.** The lecturer stresses *this word*; the dub should stress its translation. Requires word-level alignment across translation — genuinely hard, genuinely noticeable in education content (“यह **महत्वपूर्ण** है” losing its stress loses the pedagogy). Research-flagged; the flywheel’s editor emphasis-corrections are the eventual training data.
4. **Full affective transfer** (the speaker’s exact vocal emotional signature across languages) — frontier research; we adopt (Chapter 15 scouting), never build.

Emotion work is *wedge-relevant* in a way multi-speaker isn’t: a monotone dub of an enthusiastic teacher is a worse product even with one speaker. Rungs 1–2 are therefore fair game for near-term quarters; the MOS naturalness axis (Chapter 16 §16.1.3) is their gate.

18.4 What this chapter refuses to do yet

No multi-speaker marketing promises before rung 2 ships with DER gates; no “emotion AI” claims ever (we preserve a speaker’s delivery, we do not infer feelings — the distinction matters both scientifically and under Chapter 5’s bright line); no film/entertainment vertical commitments while the wedge is unwon (Chapter 2 §2.3’s expansion order stands).

Key takeaways

- The single-speaker assumption is acceptable in code and fatal in schema: `speaker_id` (and an emotion tag) enter the segment schema now, at a cost of columns, not migrations.
- Multi-speaker ladder: diarization (DER-gated, editor-verified), per-speaker voices (per-speaker consent — the charter anticipated this), per-face lip sync (research-flagged, not wedge work).
- Emotion is four distinct problems: reference-selection carryover (cheap, now), segment style tags (schema now, models as available), emphasis alignment (hard, flywheel-fed), full affective transfer (adopt from the field).

- Diarization errors and voice mis-assignment are catastrophic-class failures: they route to human review, never silently to production.

Questions founders should ask

1. Does the segment schema carry `speaker_id` and emotion tags yet — or are we re-baking the demo's assumption into the service?
2. What fraction of inbound customer content is actually multi-speaker? (Measure it in real uploads before funding the ladder.)
3. Is the cloning reference-sample selection still "seconds 5–15, hope for the best"? Rung 1 of emotion is an afternoon of work.
4. Are we marketing anything this chapter's gates haven't cleared?

Future research topics

- License-gate and DER-benchmark the current diarization field (pyannote and successors) on Indic education audio — accent and code-switch robustness is unstudied territory.
- Prototype emotion-tagged TTS with whichever Chapter 15 TTS finalists accept style controls; MOS-test against untagged baseline.
- Investigate active-speaker-detection models' licenses now (cheap scouting) so rung 3's feasibility is known before a customer asks.

Chapter 19: Experimentation & A/B Framework

Chapters 15–18 generate a constant stream of candidates: new models, isochrony rungs, emotion tags, parameter changes. This chapter is the process wrapper that decides which candidates win — and it begins with an honest correction to the founding brief, which asked for "Automatic A/B Testing" in the web-startup sense. **Traffic-split A/B testing is mostly the wrong tool for us**: our outcomes are quality scores computable offline, our traffic is small, and shipping a worse dub to half of a customer's students to learn something we could have learned on the benchmark set is both bad science and bad ethics. Our framework is therefore offline-first, shadow-second, live-split last and rarely.

19.1 The experiment hierarchy

Cheapest decisive method wins; escalate only when the cheaper level can't decide:

1. **Offline evaluation** (decides ~80% of questions). Run candidate vs. incumbent on the benchmark set (Chapter 16 §16.2); compare score vectors and CPDM. Model swaps, parameter changes (`beam_size`, rate caps), quantization trades — all decidable here, in hours, harming no one.
2. **Shadow runs** (decides most of the rest). Candidate processes real production inputs alongside the incumbent; outputs stored, scored, spot-checked by the linguist — never delivered (Chapter 13 §13.2). Catches what golden sets miss by construction: the distribution shift between our curated 50 clips and whatever customers actually upload. Per-stage artifacts make shadows nearly free; the standing rule is that **any promotion (Chapter 15 §15.5 step 5) shadows for a meaningful sample first**.
3. **Human preference tests.** When automatic scores disagree with each other or the stakes are perceptual (voice naturalness, emotion tags), run calibrated preference panels (Chapter 16 §16.4's rubric discipline) on shadow outputs.
4. **Live experiments** (rare, opt-in). Only when the question is genuinely about *user behavior* rather than output quality — e.g., does the editor's re-timing suggestion UI get used? Live splits on *dub quality* require the affected customer's informed opt-in ("beta voice program"), because students are not experimental subjects the institute didn't consent for. This is the Chapter 5 posture applied to experimentation.

19.2 The experiment record

Every experiment, one lightweight record (a sibling of the decision record, in `docs/experiments/`): hypothesis with a falsifiable claim ("IndicTrans2 ≥ current translation adequacy at ≤ cost, on benchmark v3"), method level (1–4), the pre-registered decision rule ("promote if adequacy within 1 point and isochrony improves"), result, and decision. Pre-registering the rule is the part that matters: deciding *after* seeing results which metric counts is how motivated reasoning ships regressions with a straight face. Failed experiments file the same record — the registry's archived scores (Chapter 15 §15.5 step 6)

and the experiment log together are why the same dead end doesn't get explored twice a year apart.

19.3 Velocity: the real constraint

A framework this clean can still be slow, and slow experimentation quietly becomes no experimentation. Standards that protect velocity:

- **One command.** `run-eval <candidate-adapter> <benchmark-version>` produces the full score vector and CPDM delta. If benchmarking requires manual steps, benchmarks won't happen under deadline — automation of evaluation *is* research infrastructure (the founding brief's "how experimentation happens" answered concretely).
- **The 20% reserve covers experiments too** (Chapter 3 §3.5): candidate evaluation is maintenance of the model layer, not a luxury.
- **Timeboxes over milestones for research-flagged work.** Isochrony rung 3 or emotion prototypes get "two weeks, then a record with a recommendation" — research that can't report in two weeks reports *that*, and the founder decides on renewal. This keeps frontier work from becoming an unbounded tax on a five-person company.
- **Anyone can nominate; the suite adjudicates.** The intern's model nomination and the founder's get the same `run-eval` treatment (Chapter 3's challenge duty, mechanized).

19.4 How experimentation stays innovative

The founding brief asked how the company remains innovative. Not by innovation theater (hackathons, "10% time" that dies at the first deadline) but structurally: the flywheel keeps surfacing *real* failures ranked by frequency (Chapter 20), the frontier golden tier turns them into reproducible challenges (Chapter 16 §16.2), the scouting pipeline keeps the candidate stream full (Chapter 15 §15.5), and this chapter keeps the cost of testing a candidate near zero. Innovation at SyncDub is a *pipeline property* — low friction from "what if" to scored answer — not a calendar event. When the friction rises, that is the innovation incident worth a postmortem.

Key takeaways

- Offline-first: the benchmark set decides most questions in hours; shadow runs catch distribution shift; calibrated panels settle perceptual calls; live splits are rare and opt-in — students are not unconsented test subjects.
- Every experiment pre-registers its decision rule; failed experiments file records too, so dead ends stay dead.
- Velocity is the guarded resource: one-command evaluation, timeboxed research, reserve-funded candidate testing, open nomination.

- Innovation is the low-friction path from hypothesis to scored answer, fed by real production failures — not an event on a calendar.

Questions founders should ask

1. How long does it take today to get a full score vector for a new model candidate? (If the answer involves a person's afternoon, `run-eval` is overdue.)
2. What was the last experiment whose pre-registered rule said "don't promote" — and was it obeyed?
3. Are shadow runs actually preceding promotions, or has deadline pressure made courage the method again?
4. What's the current hypothesis backlog, and does its ranking reflect correction-frequency data or the most recent customer call?

Future research topics

- Build `run-eval` as part of the first sovereignty swap (Chapter 15's translation bake-off is its natural first user).
- Define the shadow-sample-size heuristic per stage (how many real jobs before promotion is confident) using early score-variance data.
- Design the "beta program" consent language with Chapter 5's owner so live experiments have a lawful, honest home before anyone wants one urgently.

Chapter 20: Data Strategy & the Correction Flywheel

Chapter 4 named the correction-data flywheel the company's primary moat. This chapter is its engineering: what data we capture, under what rights, in what shape, and how it converts into quality. The one-line thesis: **SyncDub's most valuable production line is not the dubbing pipeline — it is the correction pipeline that runs whenever a human touches a dub.**

20.1 What a correction is

Every editor action on a job is an event: `(job, segment, stage, before, after, action_type, editor, timestamp, rationale?)`. Action types map to pipeline stages — transcript fix (ASR failure), translation fix (MT failure), re-timing (isochrony failure), voice/gender override (profiling failure), emphasis/style change (emotion failure), rejection-and-regenerate (compound failure). The mapping is the point: **each correction is a labeled example of a specific stage failing on a specific input**, which is exactly the supervision signal that eval sets, triage models, and fine-tunes are made of. Freetext "fixed some stuff" editing captures none of this; hence the standing rule (Chapter 4 §4.2) that the editor's structured capture is non-negotiable from v1 — the editor UI (Chapter 21) is downstream of this schema, never the other way around.

20.2 Rights: the flywheel runs on consent

The moat cannot be built on quietly repurposed customer content — that is both a Chapter 5 violation and a time bomb under any acquisition diligence. The rights architecture:

- **Corrections-as-data is a contract term, not a discovered behavior.** Customer agreements state plainly: correction events (the deltas, the text) may be used to improve the service; the underlying *media* may not, absent a separate opt-in. Text deltas are the flywheel's fuel anyway — this default gets us most of the value at a consent level customers readily accept.
- **Media opt-in is bought, not assumed.** Voice/video used for training or as frontier-tier eval clips (Chapter 16 §16.2) requires explicit per-customer opt-in, typically exchanged for discount — the education wedge is unusually amenable ("help us make Marathi dubbing better for everyone" is a mission their institutions share).
- **Aggregates are ours; provenance is tracked anyway.** Statistical aggregates (error rates by content type, syllable-rate statistics) carry no customer content and flow freely into research. But every dataset row retains its source-consent lineage, because "which rows may we still use?" must survive any single customer's departure or deletion request (Chapter 12 §12.3 — deletion propagates to derived datasets whose consent died with the account).

20.3 From corrections to quality (closing the loop)

Captured corrections convert through four consumers, cheapest first:

1. **The frontier eval tier** (immediate): correction clusters become test cases (Chapter 16 §16.2). A dozen institutes' worth of corrections gives us the only benchmark in existence made of *real Indic production failures*. This consumer requires no ML at all — just the schema and curation time.
2. **Triage calibration** (early): corrections are ground truth for the automatic scorer — segments editors fixed that the scorer passed are the scorer's false negatives (Chapter 16 §16.3's meta-loop). This directly moves review economics, the margin lever of Chapter 21.
3. **Failure-ranked research** (ongoing): the correction stream ranks Chapters 17–18's backlog by observed frequency — isochrony re-timings vs. code-switch mistranscriptions vs. honorific errors — replacing anecdote-driven prioritization (Chapter 19 §19.4's ranking input).
4. **Fine-tuning** (later, gated): when volume suffices, correction pairs fine-tune stage models — translation first (before/after text pairs are the cleanest signal and MT fine-tuning is the most mature path). Gated on: enough data, a Chapter 16 suite able to prove the fine-tune helps, and Chapter 2 §2.4's boundary (narrow fine-tuning, never frontier research).

20.4 Dataset governance

Boring on purpose, because dataset chaos is irreversible in a way code chaos isn't:

- **Datasets are versioned, immutable artifacts** with manifests: source jobs, consent lineage, date range, curation rules. "The training data" with no version is as meaningless as "the code" with no commit.
- **One catalog** (a directory of manifests beside the model registry) lists every dataset, its consent basis, and its consumers. When DPDP deletion or a churned customer's rights revocation arrives, the catalog answers "what must be rebuilt?" in minutes.
- **External data enters through the license gate.** Public corpora (AI4Bharat's Indic corpora and similar) are treated exactly like models in Chapter 15 §15.5: license read, decision record, then use. "It was on Hugging Face" is not a rights basis, and dataset licensing hygiene is the difference between our corpus being an asset or a liability at diligence time.
- **The linguist owns curation quality** (Chapter 6 hire #3): inter-editor consistency, rationale sampling, taxonomy evolution. A flywheel spinning on noisy labels amplifies noise.

20.5 The cold-start honesty

Today the flywheel has zero rotations: no editor, no customers, no corrections. The strategy is not blocked by this — it is *sequenced* by it: the schema ships before the editor (this chapter), the editor ships with the first customer (Chapter 21), contracts carry the data terms from customer one (§20.2 — retrofitting them is somewhere between awkward and impossible), and the frontier tier starts mattering around the tenth institute. The only cold-

start *mistake* available is building v1 without the event schema and the contract language — everything else compounds on its own schedule.

Key takeaways

- The correction event — job, segment, stage, before/after, type — is the company's most valuable data structure; the editor exists to produce it.
- Rights by design: correction deltas usable by default contract, media only by paid opt-in, every dataset row carrying consent lineage that survives deletion requests.
- Four consumers close the loop: frontier eval cases (no ML needed), scorer calibration (moves review margins), failure-ranked research priorities, and eventually translation-first fine-tuning.
- Datasets are versioned, manifested, cataloged, license-gated, and linguist-curated — dataset chaos, unlike code chaos, cannot be refactored away.
- Cold start is fine; shipping the editor without the schema or signing customers without the data terms are the only unforced errors.

Questions founders should ask

1. Is the correction-event schema designed and reviewed yet (it was Chapter 4's research topic — has it happened)?
2. Do the draft customer agreements actually contain the corrections-as-data and media-opt-in language?
3. Once live: what fraction of correction events carry a usable `action_type` vs. falling into a generic bucket? (That ratio measures whether the editor UI is serving the schema.)
4. Can we answer "which datasets contain content from customer X?" in one query today — or would deletion be archaeology?

Future research topics

- Draft the correction-event schema jointly with the Chapter 7/8 segment schema (explicitly the same design exercise, per Chapter 7's research topics).
- Have the data-use contract clauses drafted alongside Chapter 12's DPA template — one legal engagement, both documents.
- Survey AI4Bharat and related Indic corpora licenses now, so external-data options are known before fine-tuning is on any roadmap.

PART IV — PRODUCT OS: HOW PRODUCT DECISIONS ARE MADE

Chapter 21: Product Philosophy & the Human-in-the-Loop Editor

Product philosophy in one sentence: **automation is a slider, not a switch — and the slider's handle is the editor.** The industry's failed promise is one-click perfect dubbing; the industry's expensive reality is agency work with AI garnish. SyncDub's product lives deliberately between: the pipeline does everything, a human confirms or corrects what the confidence scores flag, and every correction makes tomorrow's pipeline need less correcting (Chapter 20). This chapter defines the product principles and the editor that embodies them.

21.1 Product principles

- **Ship the review, not just the render.** The deliverable is a dub *someone stands behind*. For the wedge customer, "our pipeline plus your review" beats "our pipeline, fingers crossed" — an institute putting its brand on a lecture needs the second sentence to be false. The review step is a feature we charge for, not an apology.
- **Confidence is a UI element.** The automatic scores (Chapter 16 §16.3) surface *in the product*: segments the scorer trusts render green and collapsed; flagged segments open for attention. The customer's reviewer spends minutes, not hours, per lecture — that ratio is the product's economics and its pitch in one number.
- **The student is the user; the reviewer is the customer's face of us** (Chapter 3's value, applied). Editor UX quality is not internal tooling polish — it *is* the enterprise product surface. But output quality decisions always resolve toward the student's phone, not the reviewer's monitor.
- **No feature before its measurement.** A product surface ships with the metric that will judge it (editor: corrections per lecture and review-minutes per dubbed-minute; delivery: watch-through where hosts share it). The gauntlet (Chapter 3 §3.3) already enforces this at acceptance time; this restates it at design time.
- **Speed of feedback beats speed of pipeline.** A dub returned in an hour with instant, pleasant review beats a dub in ten minutes with clunky correction. Chapter 11 monetized our latency tolerance; this principle spends some of it on review quality.

21.2 The editor, defined by its data

Per Chapter 20, the editor is the correction pipeline's front end — its design brief is the event schema. V1 surfaces, in priority order:

1. **The segment table.** Source transcript, translation, timing bar, confidence color, per segment. Click to edit text (transcript or translation), drag to re-time, one keystroke to

flag-and-regenerate. Every action emits a typed correction event; none of this is freetext.

2. **Side-by-side preview.** Original and dub, synchronized at the selected segment (per-segment artifacts make seeking free). The reviewer never scrubs blind.
3. **The speaker/voice card.** Detected gender→voice mapping (today's Librosa pitch call, made visible and overridable — the silent auto-decision becomes an inspectable one). Cloning consent state displays here (Chapter 5 §5.2); the multi-speaker map slots in later (Chapter 18 §18.2.2).
4. **Batch actions.** Approve-all-green, regenerate-all-flagged, export. Reviewers of 40-lecture courses live in batch; a per-segment-only editor punishes exactly our best customers.

Deliberately absent from v1: waveform surgery, video editing, font/subtitle styling suites — those are adjacent products, and the moat is in corrections, not in becoming a worse Premiere.

21.3 The workflow tiers

The slider's stops, productized:

- **Auto** (API tier, Chapter 22): pipeline output as-is, confidence report attached. For integrators with their own review or risk tolerance.
- **Assisted** (the wedge default): pipeline + customer's reviewer in our editor. The confidence triage does the compression; the customer supplies the judgment. This tier feeds the flywheel fastest and prices best (Chapter 24).
- **Managed** (bounded, per Chapter 2 §2.4): pipeline + *our* editor bench (the founding linguist's network) for customers who want turnkey. Tolerated as a data-and-learning channel with an explicit ceiling — the moment managed review dominates revenue, we have become an agency and the strategy has failed; the written trigger for that alarm is managed-revenue > 30% two quarters running.

21.4 Frontend honesty

The current React app (`frontend/src/App.js`, 714 lines, one component) is the demo's upload-progress-download flow and has no architecture to carry the editor. The debt register already prices this (Chapter 14 §14.2: trigger = "first real frontend feature beyond the demo flow"). The editor is that trigger. Plan accordingly: the editor is a new frontend built on the job/segment API (Chapter 8–9), not a renovation of `App.js`; the demo flow becomes one small page of it.

21.5 Feedback loops beyond the editor

The founding brief asked for a "customer feedback loop"; ours is mostly *structural* rather than survey-based: correction events are feedback with coordinates (Chapter 20 §20.3 ranks the roadmap by them), reviewer-abandoned segments (opened, stared at,

regenerated, abandoned) mark where the editor itself fails, and the quarterly customer conversation (Chapter 6's founder job) asks one question above all: *"What did you publish anyway despite disliking it?"* — the gap between corrected and tolerated is where the next quality axis hides.

Key takeaways

- Automation is a slider; the editor is the handle, the moat's front end, and the enterprise product surface — designed from the correction-event schema outward.
- Confidence triage is the core UX: green-and-collapsed vs. flagged-and-open is what turns review from hours to minutes, and that ratio is the pitch.
- Three tiers — auto, assisted (the wedge default), managed (bounded at 30% with a written alarm) — keep us a software company that touches content, not an agency with software.
- The editor is a new frontend on the jobs API; `App.js` is the demo it replaces, not the foundation it extends.
- Feedback is structural: corrections, abandonment patterns, and the published-anyway question outrank surveys.

Questions founders should ask

1. What is review-minutes-per-dubbed-minute this month, and which Chapter 16 axis improvement would cut it most?
2. Are green segments actually trustworthy — what's the rate of customer-reported errors in segments the triage collapsed?
3. Is managed-tier revenue creeping toward the 30% alarm, and is anyone watching?
4. What did our pilot customers publish anyway despite disliking it?

Future research topics

- Paper-prototype the segment table with a real Marathi reviewer (the founding linguist's network) before building — an afternoon that prevents a quarter of wrong UI.
- Define the v1 confidence-threshold policy (what collapses, what opens) from early scorer-vs-correction calibration data (Chapter 16 §16.3's meta-loop).
- Study reviewer keyboard workflows in subtitling tools (Aegisub-class) — decades of correction-UX conventions exist; import, don't reinvent.

Chapter 22: API-First & Developer Experience

“API-first” earns its place in this handbook for a structural reason, not a fashionable one: our biggest future customers are *platforms* — ed-tech companies, LMS vendors, media asset systems — who want dubbing inside their product, not ours. The web app and the editor are one consumer of the API; they must never be a privileged one. This chapter defines what that means concretely and when the API becomes a sold product.

22.1 API-first, operationally

- **One API.** The frontend, the editor, the CLI, and paying integrators call the same `/v1` (Chapter 9 §9.4). The moment an internal surface uses a private endpoint “just for now,” the API’s completeness stops being tested by our own product — the cheapest QA we will ever have. (Today’s violation, for the record: the frontend’s reliance on the static `/outputs/final_dubbed.mp4` mount; it dies at migration Stage 4.)
- **The job is the product’s noun** everywhere: create job → poll/webhook state → fetch artifacts → submit corrections. That last item matters: **the corrections endpoint is part of the public API**, so integrators’ review UIs feed the flywheel exactly like our editor does (Chapter 20’s schema, exposed). An API-tier customer who builds their own reviewer is still spinning our moat.
- **Webhooks over polling** as soon as Stage 3 auth exists: dubbing jobs run minutes-to-hours; forcing integrators to poll is hostile. Signed webhook delivery for `job.completed` / `job.failed` / `job.review_ready`, with polling retained as the fallback.
- **Consent is API-visible.** The attestation and cloning opt-in (Chapter 5 §5.2) are required request fields, not web-form-only ceremonies — integrators inherit our trust posture in their own flows, which is precisely what an education platform’s lawyers want to see.

22.2 Developer experience standards

DX is a compounding adoption channel (gauntlet question 8) and it is cheap to do well at our size:

- **Docs are the product’s front door**: a quickstart that gets a developer from key to dubbed sample clip in under fifteen minutes, an honest reference generated from the OpenAPI spec, and a documented sample video so first calls need no content hunt.
- **Errors teach.** The Chapter 9 error envelope, plus prose: `"consent_attestation missing — see docs/consent"` beats `400 Bad Request` by exactly one saved support ticket per occurrence.
- **One official SDK, Python first** (our audience: ed-tech backends and ML-adjacent integrators), generated plus hand-polished. JavaScript second. Nothing else until asked twice by paying customers.
- **A sandbox that costs us nothing and them nothing**: test keys process only the documented sample clips (pre-computed results served as if fresh) — full API

mechanics, zero GPU spend, zero consent ambiguity.

- **Status page and changelog from the first external key.** API trust is mostly communication hygiene: what changed, what broke, what's deprecated, with the Chapter 9 versioning promise visibly kept.

22.3 Rate limits, quotas, and metering

Every key carries per-tier quotas (jobs/day, minutes/month) enforced at the API layer — this is the billing meter (Chapter 24) and the abuse throttle (Chapter 5 §5.5's volume-anomaly signal) in one mechanism. Metering is by *dubbed output minute*, matching pricing, so a customer's invoice is explainable from their own API logs. Design rule: **429s must be honest and predictable** — documented limits, remaining-quota headers, no silent throttling; surprise rate limiting burns integrator trust faster than low limits do.

22.4 When the API becomes a product

Sequencing, because API-as-product costs support and stability promises that must be bought consciously:

1. **Now → first customers:** the API exists (it's how our own frontend works) but is sold only as part of assisted-tier deals. No public keys.
2. **After the sovereignty swaps + Stage 4** (legal outputs, real storage): design-partner keys — three to five integrators, hand-held, half-price, in exchange for DX feedback and case studies. Their friction list is the DX roadmap.
3. **Public self-serve keys** only when: the benchmark is publishable (Chapter 16 §16.5 — API customers *will* benchmark us unsupervised, so we go public when unsupervised evaluation flatters us), the sandbox exists, and support load per key is known from the design partners.

22.5 What API-first does not mean

Not building for imaginary integrators ahead of the wedge (the web product serves the wedge; the API's first consumer is us); not feature-parity paralysis (the editor may ship UI affordances before their API equivalents — but the *data* they produce is always API-shaped); not marketplace/plugin dreams (Chapter 2's platform ambitions activate after Indic leadership, not before; the founding brief's "Marketplace Strategy" is parked exactly there).

Key takeaways

- API-first means our own products are ordinary API consumers — the frontend using the same `/v1` as integrators is the cheapest completeness test available.
- The corrections endpoint is public: integrators' reviewers feed the flywheel too.

- DX investment order: fifteen-minute quickstart, teaching errors, Python SDK, zero-cost sandbox, status/changelog hygiene.
- Metering equals pricing equals quota — one mechanism, explainable invoices, honest 429s.
- Sequencing: internal API now, design partners after the swaps make outputs legal, self-serve only when the public benchmark and sandbox make unsupervised evaluation safe.

Questions founders should ask

1. Can our own frontend run entirely on documented `/v1` endpoints today? Every gap is a lie the API tells integrators.
2. How long does key-to-dubbed-sample actually take a fresh developer? (Time it with the next hire’s onboarding.)
3. Are the design partners’ friction lists actually driving the DX backlog, or are we polishing what’s fun?
4. Would an unsupervised integrator’s benchmark of us today produce a case study or a churn note?

Future research topics

- Write the OpenAPI spec early (already a Chapter 9 research topic) — it is simultaneously the Stage 2–3 design doc, the docs source, and the SDK generator input.
- Prototype the webhook signature scheme alongside Stage 3 auth so it isn’t retrofitted onto shipped integrations.
- Identify the three ideal design partners from the wedge’s adjacent platforms (Marathi ed-tech, LMS vendors serving Maharashtra institutions) — a Chapter 25 conversation with a Chapter 22 deliverable.

Chapter 23: Enterprise Readiness

“Enterprise” in our market means education boards, university systems, large coaching chains, media houses, and government skilling programs. They buy differently: procurement processes, security questionnaires, contracts with SLAs, invoices instead of cards, and — in our category specifically — *someone whose job depends on the dub not embarrassing the institution*. This chapter defines what enterprise-ready means for SyncDub and, more importantly, **when each piece gets built: on a named deal’s demand, not on speculation.** Premature enterprise-building is how startups drown the wedge product in checkbox features; late enterprise-building loses one deal, teaches the lesson, and the second deal closes.

23.1 What our enterprise buyers actually require

Ranked by how often it decides the deal, per the realities of Indian institutional procurement:

1. **Accountability for output quality.** The assisted tier’s review workflow (Chapter 21 §21.3) with named reviewer sign-off *is* the enterprise feature — an audit trail showing a human approved each published lecture. We get this almost free from the correction schema; sell it as governance, because that is what it is.
2. **Trust documentation.** Consent chain, provenance ledger, misuse policy, deletion story (Chapters 5, 12) — packaged as a plain-language trust dossier a procurement officer can forward. The engineering exists by design; the *packaging* is a marketing-week task that closes deals disproportionate to its cost. Government content programs in particular will make synthetic-media governance a tender requirement before most vendors can spell C2PA; being early here is the Chapter 4 trust moat converting into revenue.
3. **Data residency and confidentiality.** In-India processing (the Chapter 11/12 joint region decision), a signable DPA (Chapter 12’s research topic), and honest answers about model training on their content (Chapter 20 §20.2’s contract terms — our default *is* the answer they want).
4. **Commercial plumbing.** GST invoices, purchase orders, NEFT payment, multi-year quotes, and rate-contract formats for government. Zero engineering, real close-rate impact; a founder-afternoon with an accountant, not a roadmap item.
5. **SSO and role separation.** Reviewer vs. admin roles arrive with the editor’s tiers naturally; SSO (Google Workspace covers Indian education overwhelmingly) waits for the first deal that names it — it is a Stage 3 auth extension, priced into that deal.
6. **SLAs.** Per Chapter 13 §13.5: an SLA is a priced promise. Our battery: job-completion time targets (not uptime theater — buyers care when the semester’s lectures land), support response tiers, and a re-run-free remedy for quality incidents. Signed only at volumes where the roster answers 2 a.m. (or the price funds the roster that does).
7. **Certifications.** SOC 2/ISO 27001 on a named deal’s written requirement only — with Chapter 12’s habits as pre-paid evidence, certification is months of paperwork, not

years of rebuild. In Indian education procurement it is asked about more often than actually required; the trust dossier usually satisfies.

23.2 The enterprise feature gate

Standing rule extending the gauntlet (Chapter 3 §3.3): **an enterprise feature is built when a named account with a plausible close names it as blocking, and its cost is priced into that deal.** Speculative enterprise work fails gauntlet question 10 by default. The one standing exception is anything in the trust dossier — that is moat work (Chapter 4) that happens regardless of deals.

Corollary — the **enterprise readiness ledger**: a one-page list of known future asks (SSO, SOC 2, on-prem, VPC peering) each marked "awaiting named deal," with rough cost. When a prospect names one, the conversation is a lookup, not a scramble, and the deal prices it knowingly.

23.3 On-prem and air-gapped, answered in advance

The wedge's government edge will eventually ask for on-premise deployment — the `Project_Specifications.md` future-scope already intuited this ("offline translation… full air-gapped security"). The prepared answer: the sovereignty swaps (Chapter 15) move us toward self-hostable open models *anyway*; the job-system architecture (Chapter 8) containerizes *anyway*; therefore a single-tenant "SyncDub appliance" is a packaging problem, not a rewrite — but an expensive packaging problem (support, upgrades, GPU procurement on their side), so it is quoted at a price that reflects a forked operational burden, and only for deals that justify a dedicated engineer-quarter. Until then: in-India cloud with residency guarantees answers 90% of the underlying concern at 10% of the cost.

23.4 Support standards

Support is tiered like everything else: self-serve docs and status page (all tiers, Chapter 22 §22.2); named-contact email with response targets (assisted tier); a shared channel and quarterly review call (enterprise). Two disciplines keep it sane: **support tickets are correction data too** — quality complaints route into the same triage that ranks the research backlog (Chapter 20 §20.3) — and **the founder reads every ticket until hire #5 exists** (Chapter 6), because at this stage support *is* customer discovery wearing a different hat.

Key takeaways

- Our enterprise buyers purchase accountability, trust documentation, residency, and commercial plumbing before they purchase features — and most of that we get by packaging what Chapters 5, 12, 20, 21 already build.
- The gate: enterprise features are built on named deals and priced into them; the trust dossier is the standing exception because it is moat work.

- Keep an enterprise-asks ledger so prospects' requirements meet prepared answers with known costs.
- On-prem is a foreseen packaging problem made cheaper by decisions already taken (open models, containerized jobs) — quoted expensively, built reluctantly, never before a deal that funds it.
- SLAs are priced promises; support is customer discovery; every quality ticket feeds the flywheel.

Questions founders should ask

1. Does the trust dossier exist as a forwardable document yet? It is the highest-leverage sales asset this chapter names.
2. What is on the enterprise-asks ledger right now, and did anything get built this quarter without a named deal attached?
3. Can we issue a GST invoice against a purchase order today without improvisation?
4. Which prospective deal is closest to naming SSO, SOC 2, or on-prem — and is the priced answer ready?

Future research topics

- Draft the trust dossier from Chapters 5 and 12's commitments (a writing task, not an engineering one) and test it on a friendly procurement officer.
- Research Maharashtra government empanelment/tender norms for content-services vendors — the wedge's government edge has formal entry rules worth knowing early.
- Estimate the true cost of a single on-prem deployment (support hours, upgrade path, their-GPU sizing) so the §23.3 quote is grounded before it is ever needed.

Chapter 24: Pricing, Packaging & Unit Economics

Pricing is a product decision — it tells the market what we are — and a strategy decision — the corridor between human-dubbing costs and our CPDM *is* the business (Chapter 2 §2.3). This chapter sets the pricing philosophy, the packaging that expresses the tiers, and the unit-economics discipline that keeps the corridor real. Numbers here are placeholders with reasoning attached; the reasoning is the standard, the numbers get replaced by measurement.

24.1 The corridor

Three reference points define our room:

- **Ceiling — human dubbing.** Professional Indic dubbing runs at studio rates that price most education content out entirely (which is why the content sits untranslated — the market's existence proof). Whatever the current quote per finished minute is in Mumbai studios, that is the ceiling's order of magnitude: *thousands of rupees per minute*.
- **Floor — our CPDM** (Chapter 11), *honestly loaded*: compute, storage, licensed model fees post-swap (the fake zero made real, §11.1), review minutes, support. Unmeasured today; the first measurement is scheduled work, and until it exists every price is a guess wearing confidence.
- **Reference — global competitors' INR-translated pricing** (Rask/HeyGen-class subscriptions). Relevant mostly as the anchor *international-minded* customers arrive holding; our wedge buyers compare against the ceiling and against ₹0-and-stay-undubbed, not against Silicon Valley SaaS tiers.

The pricing thesis: **price per dubbed minute, an order of magnitude below human dubbing, at a healthy multiple above loaded CPDM** — concretely, target a CPDM:price ratio of 1:4 or better, reviewed monthly (Chapter 11 §11.1), because model fees, GPU markets, and review ratios will all move under us.

24.2 The unit, defended

The billing unit is the **dubbed output minute** (matching the API meter, Chapter 22 §22.3). Rejected alternatives, recorded so they stay rejected: per-video (punishes short content, invites 3-hour "videos"), per-seat (our value scales with content volume, not humans logged in — seat pricing invites one-login institutes), flat unlimited (an invitation to be someone's free GPU farm), credits-with-expiry (short-term revenue, long-term resentment; wedge institutions budget annually and hate breakage). Per-minute is explainable in one sentence, maps to cost, and survives procurement scrutiny.

24.3 Packaging: the tiers price the slider

Packaging mirrors the workflow tiers (Chapter 21 §21.3), so the price list *teaches the product*:

- **Auto (API)**: lowest per-minute rate; confidence report included; no review. For integrators (Chapter 22 §22.4's sequencing gates when this is even available).
- **Assisted** (the wedge default): mid rate; editor seats included free (the editor feeds the flywheel — charging for it would tax our own moat); volume-tiered per-minute discounts, because the 400-lecture institute is exactly whom we want and volume improves both flywheel and margin.
- **Managed**: highest rate, quoted per project; priced to *discourage* all but strategic cases (the 30% agency alarm, Chapter 21 §21.3).
- **Enterprise wrapper** on any tier: residency, SLA, SSO, invoicing — priced as the Chapter 23 ledger dictates, on named asks.

Cloning as premium: voice cloning carries a per-minute premium over stock neural voices — it costs more (heavier synthesis, consent workflow) and is worth more (the teacher's-own-voice value, Chapter 2 §2.3.4). The premium also usefully throttles cloning to customers serious enough to complete consent properly.

The free tier is a demo, not a tier: a few minutes of watermarked-output trial (visible watermark — the honest kind, Chapter 5 §5.3), enough to prove quality on *their* content, structurally incapable of being a production freeloader. In a GPU-cost business, generous free tiers are venture-subsidized marketing we cannot and should not copy.

24.4 Unit-economics discipline

- **The weekly number is CPDM:price by tier.** Margin erosion is quiet and specific: one enterprise deal's review-heavy content, one model swap's fee structure, one customer's pathological audio. Per-tier (eventually per-account) unit tracking finds it while it's a conversation, not a crisis.
- **Review minutes are the swing variable.** At target, assisted-tier review cost falls as triage improves (Chapter 16 §16.3.1) — this is the *margin expansion story*: the flywheel doesn't just improve quality, it widens gross margin every quarter the scorer gets better. That sentence is also the investor narrative's economic core (Chapter 26).
- **Discounts buy something.** Volume discounts buy flywheel data; design-partner discounts buy case studies and DX feedback (Chapter 22 §22.4); media-opt-in discounts buy training rights (Chapter 20 §20.2). A discount that buys nothing is just margin donated to negotiation theater — every non-list price names what it purchased, in the deal record.
- **Price changes are experiments with records** (Chapter 19's discipline applied): hypothesis, cohort, decision rule, result. Grandfather existing customers by default — wedge institutions run on annual budgets and trust; repricing them mid-year spends the moat to make a quarter.

24.5 The honest sequence

Today, with unlicensed models and no consent flow, **we cannot charge anyone anything** (Chapter 1 §1.3) — pricing work is therefore: (1) measure CPDM on the current pipeline now (it bounds every plan), (2) complete the sovereignty swaps that make revenue legal, (3) pilot-price the first three institutes individually against measured CPDM with the 1:4 target, (4) publish the price list only after three pilots confirm the review-minutes assumption — the number most likely to surprise us.

Key takeaways

- The corridor — human-dubbing ceiling, loaded-CPDM floor — is the business; target CPDM:price of 1:4+, reviewed monthly.
- Bill per dubbed output minute; the rejected alternatives stay rejected for recorded reasons. Packaging mirrors the automation slider; cloning is premium; the free tier is a watermarked demo.
- Review minutes are the swing variable, and triage improvement is the margin-expansion story — the flywheel compounds economically, not just qualitatively.
- Every discount purchases something named; every price change is a recorded experiment; pilots precede price lists.
- Nothing is chargeable until the swaps land — which makes CPDM measurement and the Chapter 15 agenda the *revenue* roadmap, not just the legal one.

Questions founders should ask

1. What is loaded CPDM this week, by tier, and is any tier’s ratio drifting below 1:4?
2. What did each active discount purchase, and did we collect it (the case study, the data rights, the feedback)?
3. What are actual review-minutes-per-dubbed-minute in the pilots vs. the pricing assumption?
4. Is anything being sold today that Chapter 1 §1.3 says cannot legally be sold?

Future research topics

- Get three real quotes for human Marathi dubbing of a one-hour lecture (the ceiling deserves data, not folklore).
- Model the assisted-tier P&L at 10 / 100 / 1,000 dubbed hours per month using measured CPDM — the spreadsheet that answers “when does this fund itself?”
- Survey wedge buyers’ budget cycles and procurement thresholds (the price *format* — annual contract vs. per-minute metered — may matter more than the price level in education).

PART V — BUSINESS OS: HOW THE COMPANY WINS

Chapter 25: Go-To-Market & Competitive Strategy

Strategy that never names competitors is not strategy (Chapter 1's critique of the founding brief). This chapter names them, positions against them, and sequences the market entry — with the standing caveat that competitor facts rot fast: the *analysis method* here is permanent, the snapshot gets re-verified before any decision leans on it.

25.1 The competitive map, by what each player optimizes

- **YouTube auto-dubbing** — the ambient threat. Free, automatic, improving, and distribution-integrated. What it structurally won't do: review workflows, accountability, off-YouTube delivery, institutional governance. It commoditizes *unreviewed* dubbing — which sharpens our position rather than killing it: we sell the dub someone stands behind (Chapter 21 §21.1). If a prospect is happy with auto-dub quality on YouTube, they were never our customer; the moment their brand is on the line, they are.
- **Rask / HeyGen-class SaaS** — self-serve, many-language, creator-oriented, subscription-priced. Strong products, structurally weak on Indic prosody (Chapter 2 §2.3.1), enterprise governance, and Indian price points/procurement. They win international creators; we concede that segment happily for now.
- **Papercup / Deepdub-class** — enterprise media localization with human-in-the-loop, aimed at broadcasters, priced accordingly. Closest to our *model* (they validated assisted dubbing), farthest from our *market* (Western media budgets). Watch them for product patterns, not for deal collisions — yet.
- **Sync Labs** — the Wav2Lip authors' commercial API. Simultaneously a component vendor (a lip-sync stage candidate, Chapter 15 §15.3), and proof of the stage's value. A reminder that today's supplier can be tomorrow's competitor — which is exactly why sovereignty (Chapter 7 §7.1) treats every model as replaceable.
- **Indian AI voice/speech startups** (the Sarvam-adjacent field) — the real future collision, because they share our Indic thesis. Most are voice/LLM-first, not video-workflow-first; our differentiation against them is the full localization workflow (editor, governance, lip sync) and the correction flywheel. This lane deserves quarterly re-checks, not annual.
- **Local dubbing studios** — not competitors: the ceiling (Chapter 24 §24.1), a talent network for the managed tier, and possible channel partners for media-house deals.

The positioning sentence that survives all of the above: *SyncDub is the accountable way to publish your content in Indian languages — your voice, your timing, your review, your consent chain.* Every word maps to a moat (Chapters 4, 5, 17, 21).

25.2 GTM phase 1: win the wedge by hand

First 10 customers — Marathi-market coaching institutes, university content cells, ed-tech catalogs — are founder-sold (Chapter 6 §6.1 said this is the founder's actual job; here is the job description):

- **The pitch is a pilot, not a deck**: take one of *their* lectures, return it dubbed with the review report, and sit with their reviewer for the first editing session. The demo instinct (Chapter 1) finally pointed at revenue. The competition guide's structure — hook on language access, show the magic, name the pipeline — survives; the audience changes from judges to buyers, and the pre-recorded backup becomes a pilot on their content.
- **Sell where quality tolerance and volume intersect** (Chapter 2 §2.3.2): recorded-lecture back-catalogs first — huge minute-volume, no live deadlines, patient review cycles. Perfect flywheel food.
- **Success metric for phase 1 is not revenue**: it is three institutes publishing dubbed content *under their own brand* and renewing. That proves accountability-positioning, review economics (Chapter 24's swing variable), and the flywheel's first rotations — the three assumptions everything else stands on.

25.3 GTM phase 2: repeatability and channels

After the wedge proves: productize the pilot motion (self-serve trial with the watermarked demo tier), publish the benchmark (Chapter 16 §16.5 — our loudest marketing act, timed for when we win it), activate design-partner integrations as channels (LMS/ed-tech platforms embedding via API carry us into their customer bases — Chapter 22 §22.4), and expand language pairs by *market pull* recorded in lost-deal notes, not by a language checklist's aesthetics. Marketing philosophy throughout: **publish measurements, not adjectives** — benchmark results, isochrony numbers, case studies with named institutions; in a category drowning in demo-ware hype, being the vendor with receipts is differentiation.

25.4 GTM phase 3: the moats sell

Enterprise/government scale (Chapter 23's buyers): the trust dossier leads, empanelment groundwork (Chapter 23's research topic) matures, and the flywheel's accumulated Indic depth becomes the reference-selling story ("here is our measured quality on ten thousand hours of Marathi education content — here is everyone else's"). International Indic diaspora content and non-Indic pairs stay parked until this phase funds them (Chapter 2 §2.3's expansion order, held under pressure).

25.5 Competitive doctrine

Three standing rules: **(1) Never compete on demo-wow** — we lose that fight against venture-funded polish and win on measured quality, governance, and price-at-volume; every

marketing artifact passes the “is this a measurement or an adjective?” test. **(2) Fast-follow features, never fast-follow positioning** — if Rask ships a feature our customers ask about, the gauntlet evaluates it on our terms; if a competitor pivots to “accountable Indic localization,” *that* is the emergency worth a founder-week. **(3) Watch flywheels, not features** — the competitor metric that matters is whether anyone else is accumulating Indic correction data; feature parity is noise, data compounding is signal.

Key takeaways

- The map: YouTube commoditizes unreviewed dubbing (sharpening us), global SaaS owns creators (conceded for now), Papercup validates our model elsewhere, Sync Labs proves supplier-competitor duality, Indic AI startups are the real future collision, studios are ceiling/partners.
- Position: the accountable way to publish in Indian languages — every word moat-backed.
- Phase 1 is founder-sold pilots on the customer’s own lectures; success = three institutes publishing under their own brand and renewing.
- Phase 2 scales via benchmark publication and platform channels; phase 3 lets the trust dossier and data depth sell enterprise/government.
- Doctrine: measurements over adjectives, fast-follow features but never positioning, and watch competitors’ *data accumulation*, not their changelogs.

Questions founders should ask

1. When was the competitive snapshot last re-verified against reality, and what changed?
2. Are the phase-1 pilots on track to the three-institutes-renewing bar — and if not, which assumption (quality, review economics, accountability-positioning) is the one failing?
3. What do the lost-deal notes say is the actual next language pair?
4. Is anyone else visibly accumulating Indic correction data?

Future research topics

- Run competitor trials through our own eval suite (already flagged in Chapters 2 and 17) — the single exercise that feeds positioning, benchmark validation, and honest respect for the competition at once.
- Draft the phase-1 pilot playbook (selection criteria, pilot scope, review-session script, conversion terms) as a repeatable document before hire #5 needs it.
- Map the Marathi ed-tech/LMS platform landscape for the three design-partner candidates (shared task with Chapter 22).

Chapter 26: Open Source, Community & Investor Readiness

Three decisions the founding brief asked for that share one property: they are cheap to decide now and expensive to reverse later. This chapter decides them — explicitly, with revisit-conditions, in the decision-record spirit of Chapter 3.

26.1 The open-source decision

Decision: open-core, with the moat line as the boundary. The moat map (Chapter 4) makes this almost mechanical — open what is already table stakes, keep what compounds:

- **Open (when ready):** stage interface definitions and adapters, the benchmark harness and its rights-cleared golden tier (Chapter 16 §16.5 — the benchmark *must* be open to be a referee), eval metric implementations (isochrony metrics included: we *want* the industry measuring on our axes — every published LSE or drift score anywhere reinforces our category framing), and eventually the pipeline glue that Chapter 4 already declared fake-moat.
- **Closed:** the correction schema's accumulated data and datasets (the moat itself), the triage/scoring calibrations learned from it, the editor, the routing/cost machinery, and everything trust-infrastructure (open-sourcing consent tooling invites forks that strip it).
- **Sequencing:** nothing opens before the sovereignty swaps complete — open-sourcing a pipeline whose components we can't legally ship commercially would be performative — and the first open artifact is the benchmark, because it does marketing, hiring, and moat work simultaneously.
- **Revisit-condition:** if a credible open-source Indic dubbing stack emerges from elsewhere, the calculus flips from "should we open?" to "can we afford not to lead it?" — that event reopens this record within the quarter.

Until then, the immediate housekeeping this decision unblocks: the repo gets its `LICENSE` (proprietary for now — Chapter 9 §9.1's missing file), and vendored third-party licenses stay scrupulously preserved.

26.2 Community strategy

Community for us is three concentric circles, funded in this order: **(1) The reviewer/linguist community** — the Marathi translators and educators who use or staff the editor; a small, real practitioner network (the founding linguist's orbit, Chapter 6) that feeds curation quality, managed-tier capacity, and the most credible word-of-mouth in the wedge. Unglamorous and first. **(2) The developer community** — activates with public API keys (Chapter 22 §22.4); until then, DX investment *is* the community strategy. **(3) The benchmark community** — researchers and rivals engaging with the published benchmark; success here looks like citations and submissions, and it is the only circle

where competitor participation is the *goal*. What we do not do: Discord-server theater ahead of having anything for members to do, or “developer relations” hires before developers exist (Chapter 6 §6.2’s non-hires, extended).

26.3 Investor readiness

The narrative, assembled from the handbook rather than invented for the deck — which is the point: **diligence-proof storytelling is just the truth, indexed.**

- **The story:** massive under-served market (Indian-language content access), a wedge with existence proof (education back-catalogs, Chapter 2), an economic engine with a compounding margin story (the flywheel widening gross margin as triage improves — Chapter 24 §24.4, the single most fundable sentence we own), and defensibility a VC can audit (the moat map, the benchmark, the data terms in actual contracts).
- **Milestones that map to raises:** the Chapter 25 phase gates *are* the funding gates — three-institutes-renewing de-risks seed; repeatable pilot motion + benchmark published + design partners live de-risks Series A. Raising against calendar instead of gates is how the expansion order (Chapter 2 §2.3) gets sold to the highest bidder.
- **The diligence file, maintained not assembled:** decision records, the model registry with licenses (the Chapter 1 §1.3 problems *visibly solved* — an acquirer’s counsel will look exactly there), consent/data-rights contract terms, CPDM history, the debt register. A data room that is just our normal documentation is itself a signal most startups can’t fake.
- **What we don’t do for investors:** vanity metrics (registered users of a free demo tier), GMV theater via the managed tier (the 30% alarm, Chapter 21 §21.3, exists partly for this), or roadmap promises the eval suite hasn’t blessed. The founding brief asked this handbook to resemble a world-class company’s internals; the investor corollary is that we show internals instead of theater.

26.4 International expansion, held honestly

The brief asked for an international strategy; the honest version at our stage is a *trigger list, not a plan*: diaspora education demand appearing in lost-deal notes; an Indic pair’s flywheel maturity making a structurally similar language family (the isochrony and honorific machinery generalizes) cheap to enter; or a design partner carrying us into a market we didn’t choose. Until a trigger fires, international is a standing temptation to violate the expansion order — and this section exists mainly so that refusal has a citation.

Key takeaways

- Open-core with the moat line as the boundary: benchmark and metrics open (referee status requires it), data/editor/calibrations closed, nothing before the swaps; the repo finally gets its LICENSE.

- Community funds practitioners first, developers when keys exist, benchmark-engagers as the endgame — no community theater.
- The investor narrative is the handbook indexed: flywheel-widens-margin is the fundable sentence, phase gates are the funding gates, and the data room is our ordinary documentation kept honest.
- International expansion is a trigger list; this section is the citation for saying “not yet.”

Questions founders should ask

1. Has the LICENSE file landed yet? (The cheapest item in the entire handbook to keep failing.)
2. If a lead investor asked for the data room next week, which artifact would be embarrassing — the registry, the CPDM history, or the contracts?
3. Is any raise being timed by calendar or burn rather than by phase gates, and is that a choice or a drift?
4. Which open-source revisit-condition or international trigger is closest to firing?

Future research topics

- Draft the open-source license choice for the future benchmark release (data licensing differs from code licensing — CC variants vs. Apache; a decision record when release nears).
- Study how benchmark-led credibility played out for comparable companies (the referee strategy has precedents worth mining for timing mistakes).
- Maintain a quarterly one-page “state of the gates”: phase-gate progress against Chapter 25, moat health against Chapter 4 — the standing input to both board conversations and this chapter’s revisit-conditions.

APPENDICES

Appendix A: Review Checklists

The handbook's decision rules, consolidated into the three checklists that get used at the moments decisions actually happen. Each item cites its chapter — a checklist answer that surprises you is a chapter to reread. These lists are deliberately short: a checklist longer than a screen gets skipped under exactly the deadline pressure it exists for.

A.1 Architecture Review Checklist

Run before merging any change to system structure, schemas, stages, or dependencies.

- ☐ **License gate:** every new model/dependency has a commercial-use-compatible license and a decision record. (Ch. 7 §7.1, Ch. 9 §9.5, Ch. 15 §15.2)
- ☐ **Interface discipline:** model/stage specifics stay behind stage interfaces; no caller learns a model's name. (Ch. 7 §7.1, Ch. 15 §15.1)
- ☐ **Job-system shape:** long work is durable jobs with IDs, states, per-stage artifacts — no fire-and-forget, no global files. (Ch. 7 §7.2, Ch. 8)
- ☐ **Schema conservatism:** segment/job/correction schema changes are additive, senior-reviewed, and carry `speaker_id`/tag foresight. (Ch. 7 §7.5, Ch. 18 §18.1)
- ☐ **Containment:** any shortcut is deep, not wide — invisible to callers, registered in `docs/debt.md` with a trigger. (Ch. 7 §7.3, Ch. 14)
- ☐ **Measurement attached:** performance/cost claims come with before/after numbers; CPDM impact estimated for pipeline changes. (Ch. 7 §7.4, Ch. 11)
- ☐ **Boring by default:** no new infrastructure category (service, store, queue) without a fired need; microservices only as measured outcome. (Ch. 7 §7.4)
- ☐ **Tenancy & trust surfaces:** account-scoped queries only; auth/upload/URL-signing changes carry a threat-model paragraph. (Ch. 12 §12.2, §12.4)
- ☐ **Reversibility:** the change deploys behind expand-then-contract migrations and can be reverted by redeploy. (Ch. 13 §13.2)

A.2 Product Review Checklist

Run before committing to any feature, tier, or customer promise.

- ☐ **Gauntlet passed:** ≥3 honest yeses on the ten questions; trust gate (question 10) is a hard pass. (Ch. 3 §3.3)
- ☐ **Wedge alignment:** serves Hindi→Marathi education excellence now, or has a written revisit-condition. (Ch. 2 §2.3, Ch. 3 §3.5)
- ☐ **Bright line intact:** transforms consented content only; no fabrication pathway; consent UX not weakened. (Ch. 5 §5.1–5.2)

- ☐ **Flywheel check:** does it generate structured correction data — and if it touches editing, does it emit typed events, not freetext? (Ch. 4 §4.2, Ch. 20 §20.1)
- ☐ **Student test:** improves or preserves the end-viewer's experience on a cheap phone, not just the buyer's checklist. (Ch. 3 §3.2, Ch. 21 §21.1)
- ☐ **Measurement shipped with it:** the metric that will judge the feature exists at launch. (Ch. 21 §21.1)
- ☐ **Priced honestly:** unit-economics impact estimated; discounts purchase something named; enterprise asks tied to named deals. (Ch. 24, Ch. 23 §23.2)
- ☐ **Latency tolerance preserved:** no casual real-time or turnaround promises that forfeit the batch-cost weapon. (Ch. 11 §11.2, Ch. 3 §3.3)
- ☐ **API-shaped:** the capability exists (or is planned) as a documented `/v1` surface, not a frontend privilege. (Ch. 22 §22.1)

A.3 Engineering Review Checklist

Run on every non-trivial PR; the reviewer's working list.

- ☐ **Tier identified:** is this interface-tier code (full rigor: types, contracts, tests) or adapter-interior (pragmatic)? Argue the tier, not the taste. (Ch. 9 §9.2)
- ☐ **Tests at the right layer:** logic → unit; stage behavior → contract; wiring → golden; quality claims → eval floors. Escaped-bug fixes include the test that would have caught them. (Ch. 10)
- ☐ **Errors carry context:** failures set job/stage state; no new silent `except: pass` without a justifying comment. (Ch. 9 §9.2, Ch. 13 §13.1)
- ☐ **Config centralized, secrets absent:** no import-time env mutations, nothing sensitive in git. (Ch. 9 §9.2, Ch. 12 §12.2)
- ☐ **Dependencies pinned & licensed:** new entries justified in the PR, license read. (Ch. 9 §9.5)
- ☐ **Model changes are releases:** eval floors run, version recorded on jobs, registry updated, shadow considered. (Ch. 13 §13.2, Ch. 15)
- ☐ **Debt honesty:** touching a registered-debt area either repays or extends the entry in this PR. (Ch. 14 §14.3)
- ☐ **Post-conditions & observability:** new pipeline behavior is visible in job metrics and covered by structural post-conditions where applicable. (Ch. 13 §13.1, §13.3)
- ☐ **Docs & handbook truth:** if this PR falsifies a handbook statement, the same PR fixes the handbook. (Index rule; Ch. 13 §13.4)

A.4 The meta-checklist (quarterly, founder)

- ☐ Re-read Chapters 2, 4, 24; answer every "Questions founders should ask" in writing, honestly. (Index)
- ☐ Debt register vs. reality; fired triggers funded from the reserve. (Ch. 14)

- ☐ CPDM:price by tier; managed-revenue vs. the 30% alarm. (Ch. 24, Ch. 21 §21.3)
- ☐ Competitive snapshot re-verified; anyone else accumulating Indic correction data? (Ch. 25 §25.5)
- ☐ Phase-gate and revisit-condition sweep: what fired unnoticed? (Ch. 3 §3.4, Ch. 26)

Appendix B: Claude Behavior Protocol

The founding brief asked its AI collaborator to adopt permanent rules: challenge ideas, think independently, explain trade-offs, reject poor architecture, optimize for the company rather than for lines of code. Those rules are worth keeping — for *any* collaborator, human or AI. This appendix states them as the working protocol, and a distilled operational version is installed at the repository root as `CLAUDE.md`, where AI coding sessions automatically inherit it.

B.1 The protocol

1. **Challenge before complying.** When asked to build something, first check it against the decision stack (Ch. 3), the gauntlet (Ch. 3 §3.3), and the moat map (Ch. 4). If a better alternative exists, present it with reasons before executing the original request. Agreement without examination is a defect, not politeness.
2. **Trade-offs are stated, not buried.** Every recommendation names what it costs — the axis it degrades (Ch. 16's vector), the debt it takes on (Ch. 14), the option it forecloses. "This is better" without "…at the price of" is advocacy, not analysis.
3. **Measured beats impressive** (Ch. 3 §3.2). Claims of improvement come with numbers or with the honest statement that numbers don't exist yet — in which case creating the measurement is usually the actual task.
4. **Reject poor architecture and unnecessary complexity, in that order of vigilance.** The failure mode of skilled engineers (and capable AI) is elegant machinery nobody needed: new services, abstractions, and configurability ahead of the second use case. Boring-by-default (Ch. 7 §7.4) outranks cleverness.
5. **Think in decade-scale consequences, act in shippable stages.** Long-term thinking means schemas and interfaces that survive growth (Ch. 7 §7.5, Ch. 18 §18.1) — not building the 500-engineer company's infrastructure for a team of one. Every plan decomposes into stages that ship value alone (Ch. 8's pattern).
6. **The trust layer is not negotiable, even under instruction.** Work that would breach the bright line (Ch. 5 §5.1), skip consent, weaken tenancy isolation, or ship an unlicensed component gets flagged and stopped, whoever asked for it.
7. **Improve the corpus, not just the commit.** When work reveals a handbook error, a stale chapter, an unregistered debt, or a missing test — fix or flag it in the same effort. The standing artifacts (handbook, registry, debt register, decision records) outlive any single change.
8. **Optimize for the company, never for output volume.** More code, more documents, more features are costs until proven otherwise. The best response to some requests is a shorter path, a reused component, or a reasoned "don't."

B.2 Why these rules bind humans too

Every rule above is a restatement of a chapter that already governs the humans: the challenge duty (Ch. 3), measurement discipline (Ch. 7 §7.4, Ch. 16), containment and debt honesty (Ch. 14), trust supremacy (Ch. 3, 5, 12). An AI collaborator with *lower* standards than the handbook would erode it; one with *different* standards would fork the culture. The protocol is therefore not an AI policy — it is the handbook, addressed to a collaborator who reads fast and forgets nothing between sessions except what we fail to write down.

B.3 Scope and limits

The protocol governs *how* work is done, not *what* gets decided: decisions belong to the decision records and the humans accountable for them (Ch. 3 §3.4). An AI session proposing a decision drafts the record; a human owns it. And per Chapter 6’s hiring bar — the challenge test applies in reverse: a founder who notices the AI (or any collaborator) has stopped pushing back should treat that as the defect it is and ask why.

The operational distillation of this protocol lives in `CLAUDE.md` at the repository root and is loaded by AI coding sessions automatically. Keep the two in sync: this appendix is the rationale, the root file is the instruction set.

Appendix C: Glossary

Shared vocabulary, so a new hire reads this handbook without a search engine. Terms link to the chapter that owns them.

- **ASR (Automatic Speech Recognition)** — speech → timed text. Our first pipeline stage; currently OpenAI Whisper. (Ch. 15)
- **Assisted tier** — the default product: automated pipeline plus the customer's reviewer in our editor. (Ch. 21)
- **Benchmark set** — the 30–50-clip stratified golden set with linguist references; judge of all model swaps. (Ch. 16)
- **Bright line** — "consented transformation, never fabrication": the product-defining trust boundary. (Ch. 5)
- **C2PA** — Coalition for Content Provenance and Authenticity; the standard for cryptographically signed content-origin metadata we adopt for outputs. (Ch. 5)
- **Code-switching** — mixing languages mid-sentence (ubiquitous Hindi-English in Indian lectures); a priority failure surface for ASR/MT and a target Indic-depth advantage. (Ch. 2, Ch. 16)
- **Composite dub score** — weighted roll-up of the six quality axes; for dashboards and marketing, never for decisions (the vector decides). (Ch. 16)
- **Contract tests** — the suite any implementation of a stage interface must pass; what makes model swaps mechanically safe. (Ch. 10)
- **Correction event** — the structured record of a human edit: job, segment, stage, before/after, action type. The moat's atomic unit. (Ch. 20)
- **Correction flywheel** — corrections → better evals/triage/models → higher quality → more customers → more corrections. The primary moat. (Ch. 4, Ch. 20)
- **CPDM (Cost Per Dubbed Minute)** — total marginal cost, review minutes and license fees included, to deliver one reviewed dubbed minute; infrastructure's governing metric. (Ch. 11)
- **Decision record** — one-page written record of a hard-to-reverse decision, with an explicit revisit-condition. Lives in `docs/decisions/`. (Ch. 3)
- **Decision stack** — Trust > Quality > Cost > Speed; the company's conflict-resolution order, ranked by irreversibility. (Ch. 3)
- **DER (Diarization Error Rate)** — standard metric for who-spoke-when accuracy; gates any multi-speaker feature. (Ch. 18)
- **Diarization** — segmenting audio by speaker identity. (Ch. 18)
- **DPDP Act** — India's Digital Personal Data Protection Act; our home data-protection regime (voice prints are personal data). (Ch. 12)
- **Debt register** — `docs/debt.md`: known shortcuts, what they'll break, and the observable trigger that forces repayment. (Ch. 14)
- **Frontier set** — golden-set tier grown from real production correction clusters; the benchmark no competitor can replicate without our traffic. (Ch. 16)

- **Gauntlet** — the ten-question feature-acceptance test (nine moat/adoption questions plus the trust gate). (Ch. 3)
- **Golden pipeline test** — small end-to-end fixture run asserting structural sanity (durations, non-silence, artifacts present), never byte-exactness. (Ch. 10)
- **Isochrony** — translated speech occupying the source's time; the dub-vs-voiceover difference and our central research problem. (Ch. 17)
- **LSE-C / LSE-D** — Lip Sync Error (Confidence / Distance), SyncNet-based automatic lip-sync metrics; the harness ships in `backend/Wav2Lip_repo/evaluation/`. (Ch. 16)
- **Managed tier** — we supply the reviewers; bounded at 30% of revenue by written alarm (the anti-agency rule). (Ch. 21)
- **Model registry** — per-model record of license, eval scores, cost, and operational notes; no entry, no deploy. (Ch. 15)
- **Model sovereignty** — the principle that every pipeline component is legally usable and technically replaceable behind a stage interface. (Ch. 7)
- **MOS (Mean Opinion Score)** — calibrated human panel rating (typically 1–5) for perceptual quality like voice naturalness. (Ch. 16)
- **MT (Machine Translation)** — the translation stage; swap target #1 (IndicTrans2 the first candidate) for both legal and quality reasons. (Ch. 15)
- **Post-conditions** — structural sanity checks run on every production job; violations mean `failed`, never silent delivery. (Ch. 13)
- **Review triage** — confidence-score-driven routing of segments to light vs. full human review; the mechanism that makes review cost scale sub-linearly. (Ch. 16, Ch. 21)
- **Revisit-condition** — the named observable event that reopens a decision record; what keeps decisions from fossilizing. (Ch. 3)
- **Shadow run** — a candidate model processing real inputs alongside production, outputs scored but never delivered; the standard pre-promotion step. (Ch. 19)
- **Stage interfaces** — `Transcriber`, `Translator`, `Synthesizer`, `LipSyncer` (later `Diarizer`): task-shaped contracts models are swapped behind. (Ch. 7, Ch. 15)
- **TTS (Text-to-Speech)** — synthesis stage; stock neural voices or consented voice cloning. (Ch. 15)
- **Trust dossier** — the plain-language packaging of consent/provenance/deletion commitments that procurement officers can forward; the trust moat as a sales asset. (Ch. 23)
- **Voice cloning** — synthesizing speech in a specific speaker's voice from a reference sample; consent-gated and premium-priced. (Ch. 5, Ch. 24)
- **Wedge** — the deliberately narrow entry market: Hindi→Marathi education content. (Ch. 2)
- **WER (Word Error Rate)** — standard ASR accuracy metric against reference transcripts. (Ch. 16)